O'REILLY®

Storage

Table types
Cloning
Time travel

Tomas Sobotik

Basics

- Support for structured, semi-structured and unstructured data
- Always encrypted and compressed
- Stored across 3 availability zones
  - Separate power grids
  - Geographic separation
- Fully online updates and patches
- Billing = actual storage
  - Daily average Terabytes per month

Table types

	Permanent	Temporary	Transient	External
	Default table type Persistent until dropped Data protection Data recovery	Persistent Tied to a session Transitory data	Persistent until dropped Multiple users Persist Data	Persistent until removed External data lake Accessed via external stage Read only
Time travel	Up to 90 days	0 or 1 day	0 or 1 day	N/A
Fail safe	YES	N/A	N/A	N/A

View types

Standard View	Secure view	Materialized View
Default view type	Secure definition and details	Like a table
Owning Role	Owning Role	Results of underlying query stored
DDL available to role with access	Query optimizer bypassed	Auto-refreshed
		Secured Materialized Views

Materialized views

What

Store frequently used queries
To avoid wasting time and money

Results guaranteed to be up to date

Automatic data refresh

MV on tables and External Table

Simplify query logic

Enterprise edition feature

When

Large and stable datasets

Underlying data do not change
so often

Query result contains small number of
Rows/columns relative to base table

Query results contains results which
Requires significant processing

Cost

Serverless feature

Additional storage

Maintenance cost

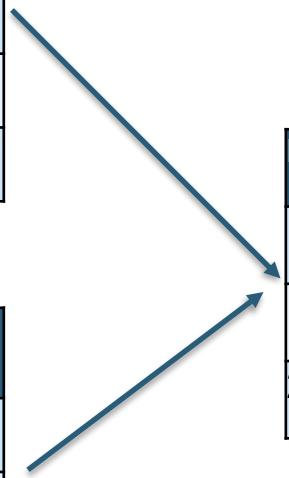
Materialized views limitations

Col 1	Col 2	Col N
	T	
	F	
	F	

Partition 14

Col 1	Col 2	Col N
	F	
	T	
	T	

Partition 28



Materialized view

Partition	Col 2	Sum
14	T	1000
14	T	2000
28	F	3000

- Only single table (no JOIN support)
- Cannot query
  - Another MV
  - View
  - User defined table function
- Cannot include
  - Window functions
  - HAVING clauses
  - ORDER BY clause
  - LIMIT clause
  - Many aggregate functions

Poll

1. What are table types in Snowflake. Mark all of them
    a) Permanent
    b) Fail Safe
    c) Transient
    d) Session
    e) External
    f) User Defined

2. External tables can be updated
    a) True
    b) False

3. Which table type can be used only by single user
    a) Permanent
    b) External
    c) Temporary
    d) Transient
    e) User Defined

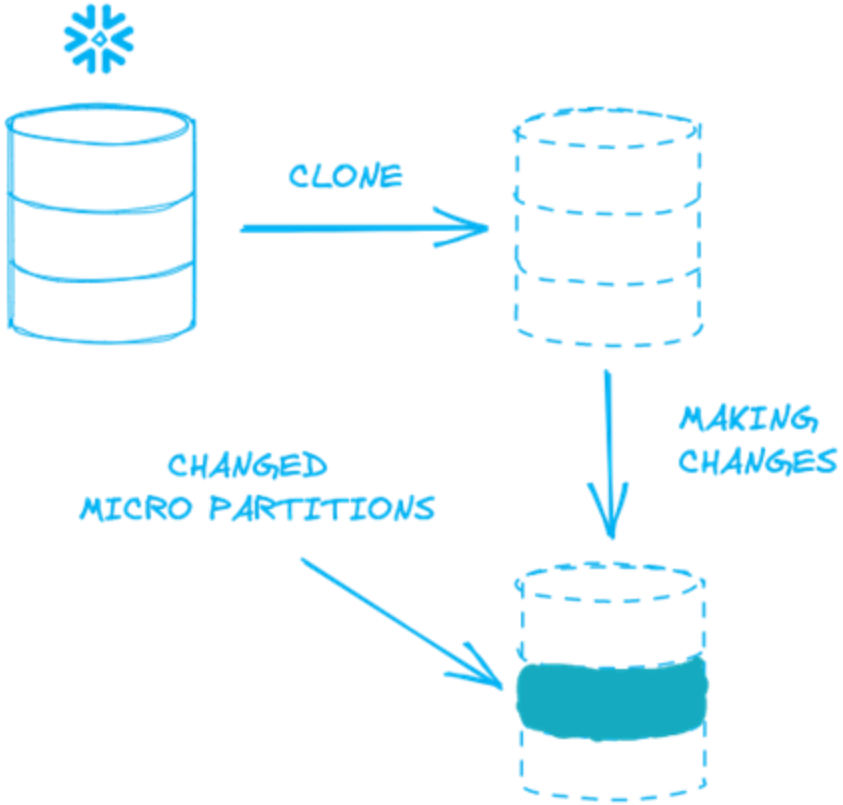
4. Secure view can get data from multiple tables
    a) True
    b) False

5. How is Materialised view refreshed - mark all possible answers
    a) By REFRESH_MV() function
    b) IN UI - refresh button
    c) Automatically

6. What can be used in MV definition. Select all correct answers
    a) JOIN
    b) GROUP BY
    c) SUM
    d) ORDER BY
    e) HAVING
    f) Another view

Zero-copy cloning

- Quickly take a snapshot of any table, schema, or database
- When clone is created:
  - Both objects share all micro partitions
  - The oldest object owns all micro partitions
  - Clone reference them
- No additional storage cost until changes are made
- Great for spinning up DEV and TEST environments
- Effective and easy “backup” solution
- Not all DB objects could be cloned

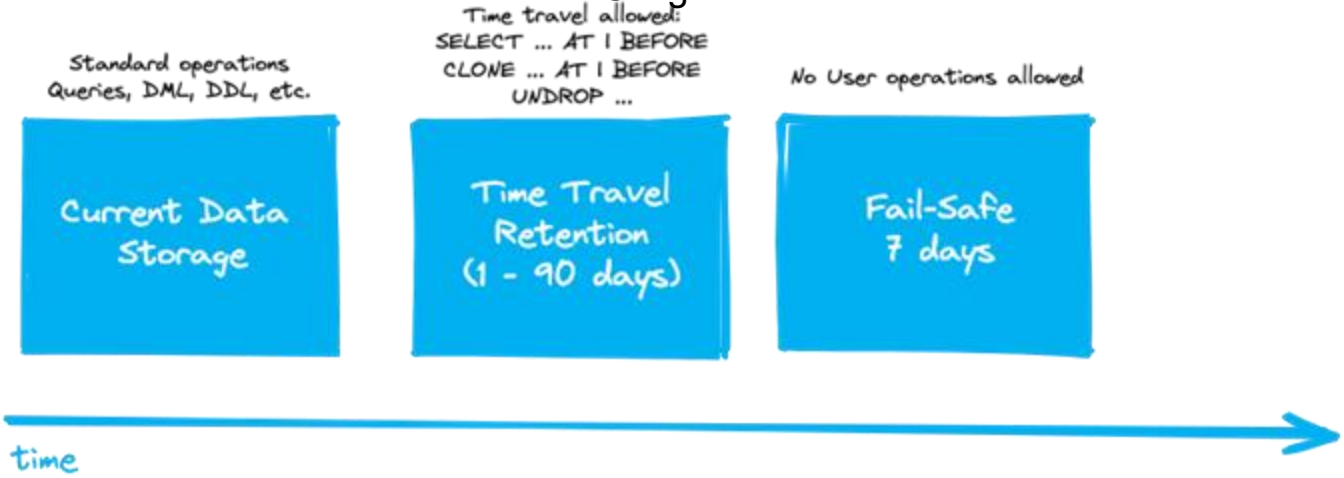


Time travel

- Access to historical data at any point within defined retention period
- Retention period 1 - 90 days
- Great for investigations when some data issue arise
- Easy way how to fix unwanted deletes or edits - UNDROP command
- Backup or restore data based on time or id
- Special SQL commands to get the data in given point of time
  - AT | BEFORE
- Requires additional storage - configure it wisely!

Fail safe storage

- Last instance to get your data in case of any disturbance in given cloud region
- Not configurable
  - 7 day retention after Time travel expiration
- To get the data you need to ask Snowflake support
- Only for Permanent tables
- Admin can view how much data fail-safe use in Usage view under Admin tab



Exercise

Let's practise the **TIME TRAVEL** and **ZERO COPY CLONING** features. Suppose we accidentally updated some column and now we need to check how data looked before the update and revert the changes back. We can use combination of TIME TRAVEL and ZERO COPY CLONING to fix it.

1. Run the following update statement to simulate the unwanted change:

```
update trips
set start_station_name =
          'Central Park S & 6 Ave TMP'
where start_station_id = 2006;
```

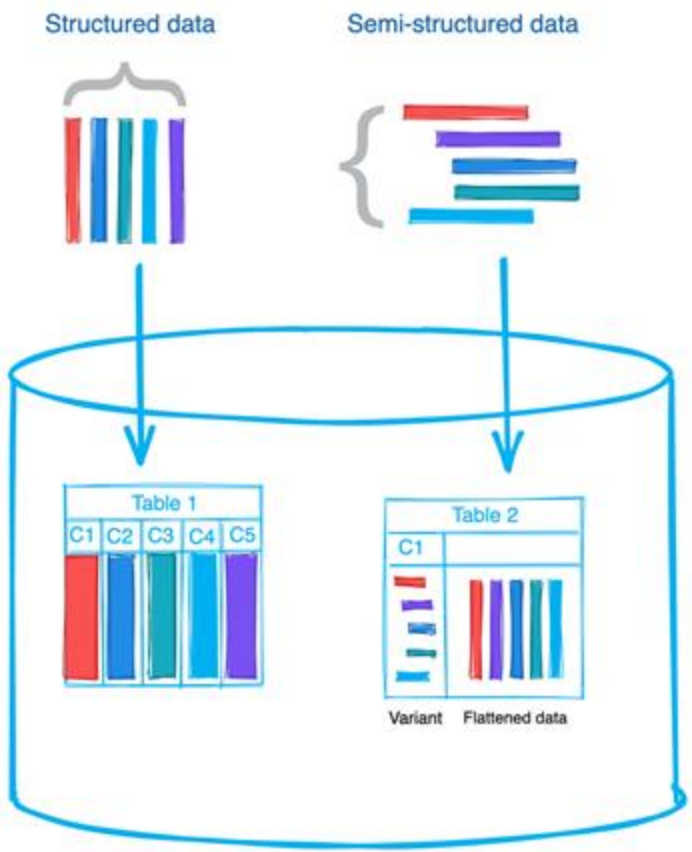
2. Use Time Travel and find all rows in table `TRIPS` where value of column `START_STATION_NAME` had the value before we did the update. Use TIME TRAVEL syntax with `AT` keyword.
3. Make a Clone of `TRIPS` table, call it `TRIPS_CLONED`. Content of the table should be from history with initial value for `start_station_name`. Again use TIME TRAVEL SQL syntax in `CLONE` statement
4. Revert the changes in `TRIPS` table. Use either Time travel or your cloned table
5. Drop the cloned table
6. If you want, you can try `UNDROP` of the cloned table but then drop it again :-)

O'REILLY®

Semi structured data

Tomas Sobotik

Overview



Native support for JSON, AVRO, Parquet, XML, ORC

Snowflake has a VARIANT data type for storing semi structured data

Stores original document as is

Traditional DBs requires complex transforms to get data into column structure

- SQL syntax is simple dot notation

When ingesting SSD, data could be columnized and metadata collected

Native support for all SQL operations

Semi structured data types

can hold standard SQL type, arrays and objects

non-native JSON types stored as strings

max length 16 MB

Usage:
Hierarchical data, direct storage of JSON, Avro, ORC, Parquet data

key-value pairs collection

Value = VARIANT

max length 16 MB

no NULL support

Arrays of varying sizes

Value = VARIANT

items accessed by position in the array

dynamic size

nested arrays are supported

JSON example

```
{
  "John Doe": {
    "address": {
      "city": "London",
      "street": "Oxford Street"
    },

    "phone": [
      "Apple iPhone",
      "Google Pixel",
      "Samsung Galaxy"
    ]

  },

  "Billy Kid "
  ....
}
```


```
[ {"John Doe": {"address": { "city": "London", "street": "Oxford Street" }, "phone": [ "Apple iPhone", "Google Pixel", Samsung Galaxy"] }, {"Billy Kid": ... } ]
```

Load options for JSON data

Store semi structured data in a VARIANT Column

You are not sure about data structure, type of performed operations or how often the structure will be changed

File has many nested levels including arrays and objects

Flatten the nested structures

Flatten if, data contains:

- dates and timestamps
- Numbers within strings
- Arrays

Depends on the use case

My recommendation: Load data as is, flatten them if needed

Casting VARIANT data

- Variant data are just strings, arrays, and numbers
- Need to be casted to proper data type otherwise they remain VARIANT object types
- There is special operator for casting ::

```
SELECT
  value:size::number(10,2) as size
  value:product_name::varchar as product
FROM my_json_data j
  LATERAL FLATTEN(input => j.src_val:data);
```

Row	Size	Product
1	10.50	A
2	11.70	B
3	12.30	C

Exercise

We are going to practise working with semi structured data. For that purpose we need to first load some JSON data into Snowflake. Let's use some sample JSON data which I have prepared

1. Create a new table for holding the JSON data

```
create table json_sample (value variant);
```

2. Copy the insert statements from file `json_sample.sql` and run them.

3. You should insert 5 records into `JSON_SAMPLE` table. Let's check it

```
select * from json_sample;
```

4. Because the JSON data has quite simple structure without nested elements and arrays, we can easily create a view on top of the JSON file with **colon notation.** Create a view `JSON_SAMPLE_VIEW` which will read data from `JSON_SAMPLE` table.

```
column_name:json_attribute_name::datatype,
```

5. Check the content of the view


```
{
  "city": "Lagunas",
  "email": "mbastiman0@washington.edu",
  "first_name": "Madelena",
  "gender": "Female",
  "id": 1,
  "ip_address": "47.136.171.159",
  "language": "Bislama",
  "last_name": "Bastiman",
  "phone": "+51 224 307 3778",
  "street": "Nevada",
  "street_number": "2"
}
```

Semi structured data functions

Array handling	`ARRAY_AGG, TO_ARRAY, ARRAY_SIZE …`
Object handling	`OBJECT_CONSTRUCT, OBJECT_KEYS, OBJECT_DELETE, OBJECT_INSERT`
JSON Parsing	`PARSE_JSON`
Extraction	`FLATTEN`
Casting	`AS_<object_type>`
Type Checking	`IS__<object_type>`

Exercise II

We are going to create a JSON structure from relational data in this exercise. It is practice for semi structured data functions like `OBJECT_CONSTRUCT` and `ARRAY_AGG`.

Please use following query as a base dataset for your JSON structure. This query gives you all the trips from station Willoughby St & Fleet St at June 9, 2018.

```
SELECT
  *
FROM
  TRIPS
WHERE
  date_trunc('day',starttime) between '2018-06-09' and '2018-06-10'
AND start_station_id = 239;
```

Please create following JSON structure. The final structure contains nested object and array as well.

```
{
  "StartStationName": << start_station_value >>,
  "day": << start_time_value >>,
  "tripDetails": {
    "duration": << trip_duration_value >>,
    "endStationName": << end_station_value >>
  },
   "userType": [ << all user_type_values for given day and start station >> ]
}
```

Exercise II – BONUS

If the given exercise was too easy for you or you would like to practise it more, you can try to do following, more complex exercise. This time create a JSON structure which will aggregate all the trips in station Willoughby St & Fleet St at June 9, 2018.

The JSON should look like this:

```
{
  "day": << start_time_value >>,
  “stationName": << start_station_value >>,
  "trips": [
    {
      "duration": << trip_duration_value >>,
      "endStation": << end_station_value >>,
      "membershipType": << membership_type_value >>,
      "userType": << user_type_value >>,
      "userbirthYear": << birth_year_value >>
    },
    {
      …. trip #2
    },
    {
      …. trip #3
    },
    ….
    {
      …. trip #N
    },
  ]
```

FLATTENING DATA

- VARIANTs usually contains nested elements (arrays, objects) that contains the data
- FLATTEN function extract elements from nested elements
- It is used together with a LATERAL JOIN
    - Refer to columns from other tables, views, or table functions

```
LATERAL FLATTEN
(input => expression [options] )
```

- Input => The expression or column that will be extracted into rows
    - It must be VARIANT, OBJECT, or ARRAY

```
SELECT
  value:size::number(10,2) as size
  value:product_name::varchar as product
FROM my_json_data j
  LATERAL FLATTEN(input => j.v:data);
```

FLATTENING DATA

- VARIANTs usually contains nested elements (arrays, objects) that contains the data
- FLATTEN function extract elements from nested elements
- It is used together with a LATERAL JOIN
  - Refer to columns from other tables, views, or table functions
  - Input => The expression or column that will be extracted into rows
    - It must be a VARIANT, OBJECT, or ARRAY

SRC.VAL

```
{
   "data":[
      {
         "size":"10.50",
         "product_name":"A"
      },
      {
         "size":"11.70",
         "product_name":"B"
      },
      {
         "size":"12.30",
         "product_name":"C"
      }
   ]
}
```

LATERAL FLATTEN

ROW	SIZE	PRODUCT
1	10.50	A
2	11.70	B
3	12.30	C

```
LATERAL FLATTEN
   (input => expression [options] )
```

```
SELECT
   value:size::number(10,2) as size
   value:product_name::varchar as product
FROM my_json_data j
   LATERAL FLATTEN(input => j.v:data);
```

FLATTEN OUTPUT

SEQ	A unique sequence number associated with the input record; the sequence is not guaranteed to be gap-free or ordered in any particular way
KEY	For maps or objects, this column contains the key to the exploded value
PATH	The path to the element within a data structure which needs to be flattened.
INDEX	The index of the element, if it is an array, otherwise NULL
VALUE	The value of the element of the flattened array/object
THIS	The element being flattened (useful in recursive flattening).

FLATTENING DATA

```
SELECT * FROM
   TABLE (FLATTEN(input => PARSE_JSON('["A","B","C"]')));
```

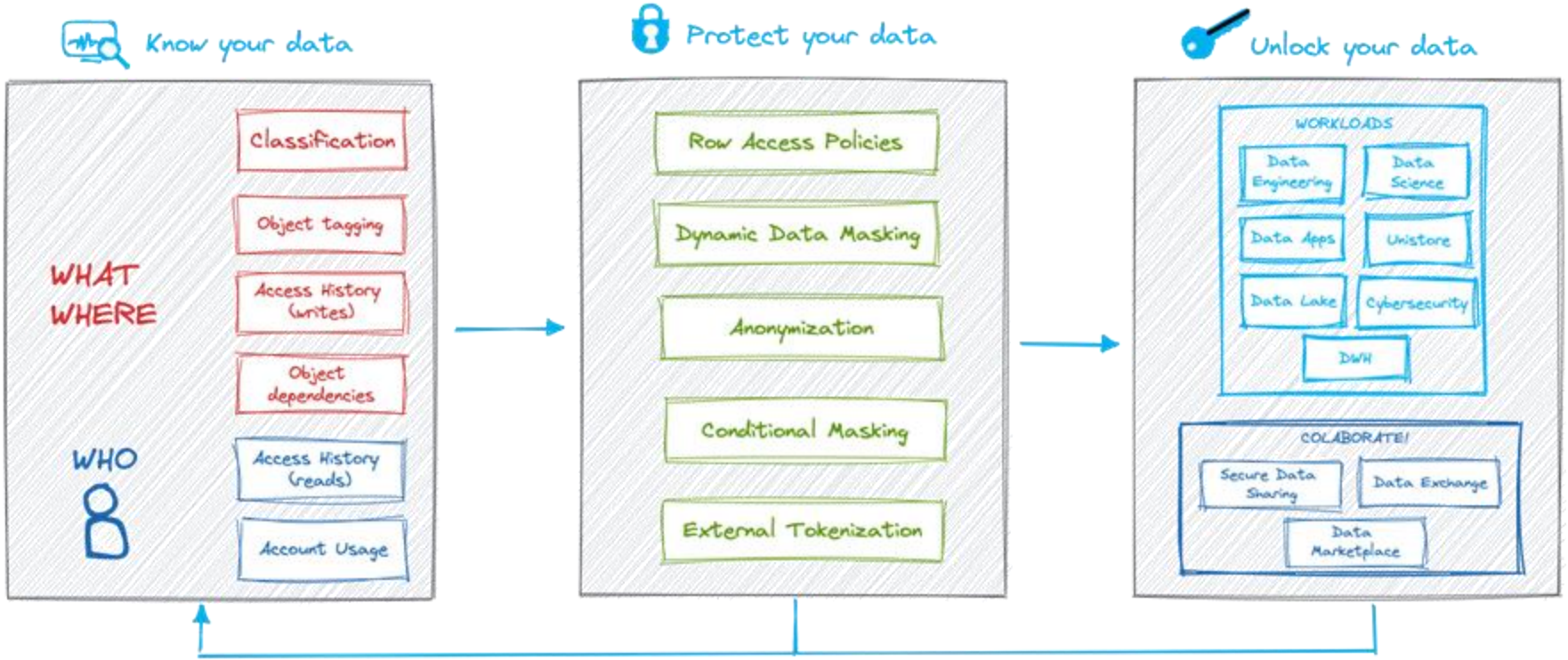
SEQ	KEY	PATH	INDEX	VALUE	THIS
1	NULL	[0]	0	“A”	[“A”,”B”,”C”]
1	NULL	[1]	1	“B"	[“A”,”B”,”C”]
1	NULL	[2]	2	“C”	[“A”,”B”,”C”]

O'REILLY®

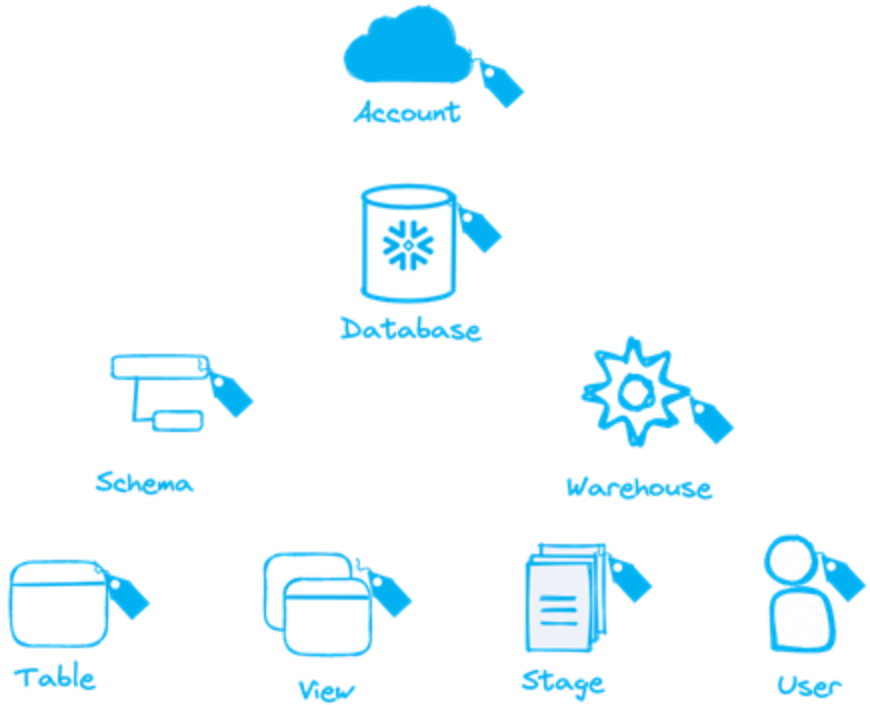
Data Governance

Tomas Sobotik

Governance overview



Object tagging



Track sensitive data and PII data

Track compute objects

Use Cases

Regulatory requirements (GDPR)

Cost management

Administration & management

Flexible privilege options

Benefits

Easy of use, tag lineage, centralized x decentralized management, sensitive data tracking and resource Usage

How to work with tags

- Create a special role for managing tags (TAG_ADMIN)
- Max 50 tags per object
- Tags are inherited based on Snowflake securable object hierarchy

1. Create a TAG

```
Create tag cost_center  comment = "cost centre tag"
Create tag purpose add allowed_values 'Pur1049', 'Pur1047'
```

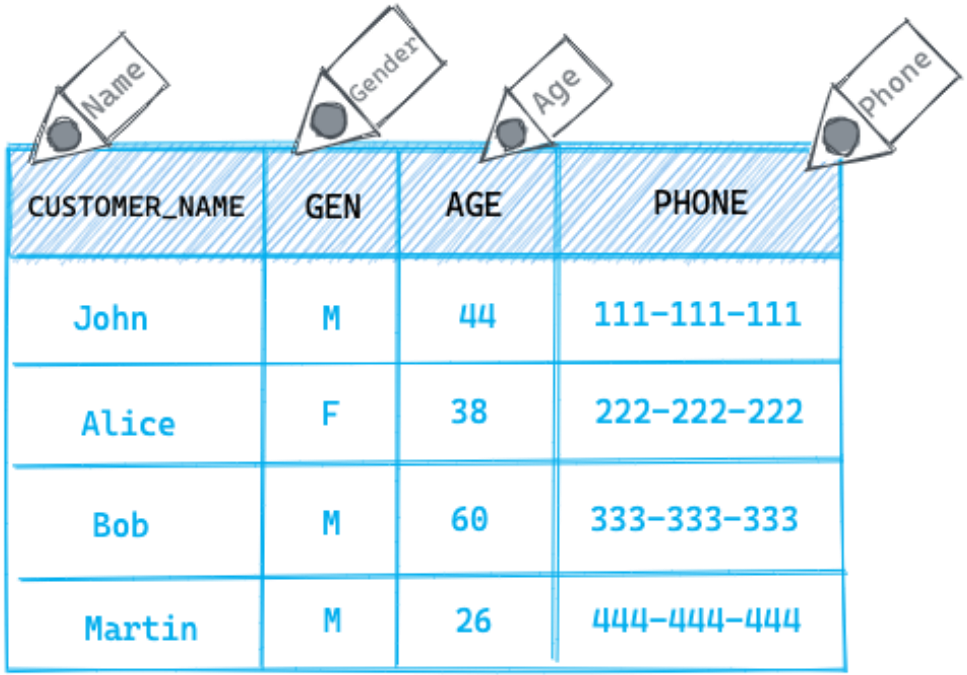
2. Assign the TAG to the DB object

```
Alter table invoices set tag cost_center = 'sales'
Alter table  employees modify_column department = 'internal'
Create table my_table with tag cost_center = 'hr'
```

Useful system views:

- ACCOUNT_USAGE.TAGS - overview about all the tags in your account
- ACCOUNT_USAGE.TAGS_OVERVIEW - details about assigned tags

Classification



Analyze table columns for sensitive personal information

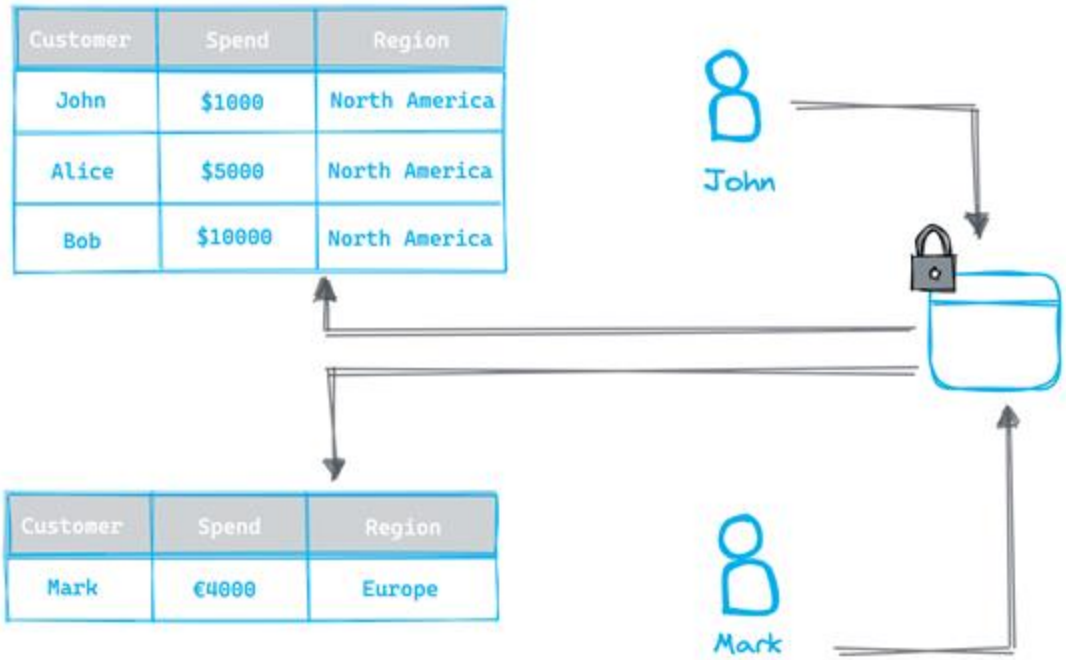
Get Recommended tags using built-in machine learning

- `EXTRACT_SEMANTIC_CATEGORIES`
- `ASSOCIATE_SEMANTIC_CATEGORY_TAGS`

Apply tags to track and audit sensitive data

Row Access Policies

Row level security



Filter unauthorized rows at query time

Authorization can be based on:

- User roles
- Mapping tables

One policy can be applied to many tables

Central management

Row Access Policies details

- Applied on query run time
- All rows and all accesses are protected (including table/view owner)
- Benefits
  - easy of use
  - change management
  - Data administration and segregation of duties

```
create or replace row access policy sales as (region varchar) returns boolean ->
  'sales_executive_role' = current_role()
      or exists (
            select 1 from sales_regions
              where sales_manager = current_user()
                and region = region
          )
;
```

Mapping table

Sales_manager	Region
John	North America
Alice	North America
Mark	Europe

Poll

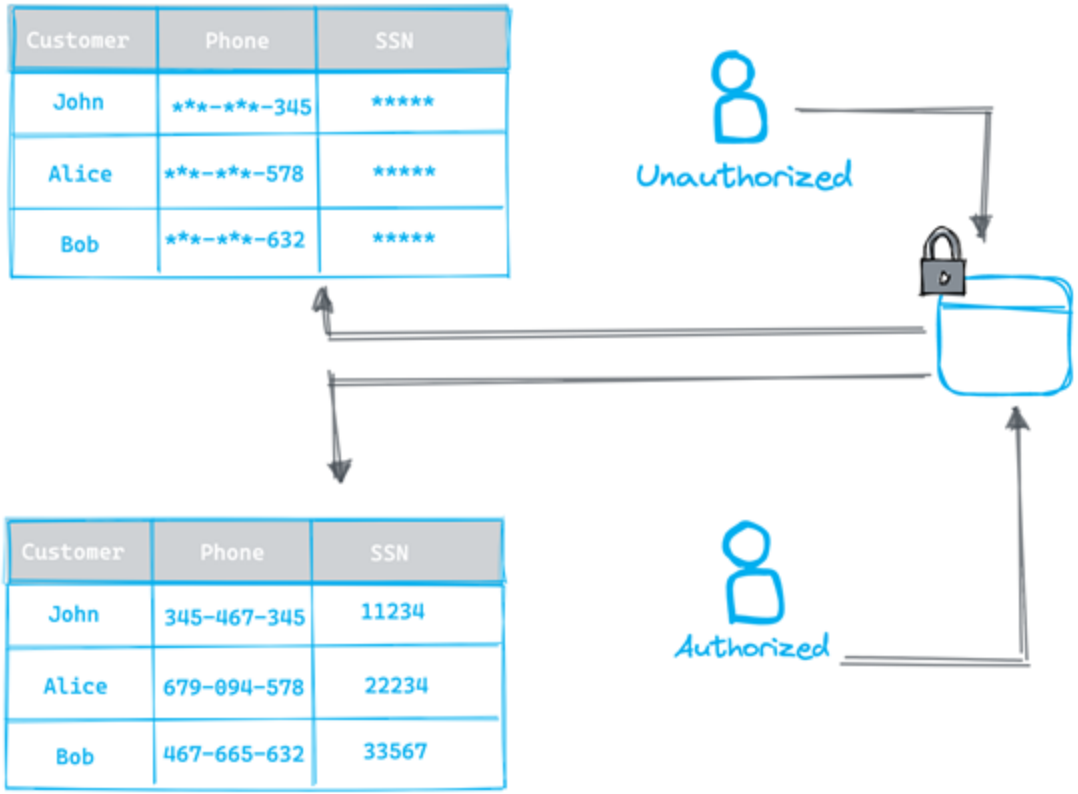
1. What is not a Snowflake Data Governance feature?
    a) Anonymization
    b) Dynamic Data Masking
    c) Column Access Policies

2. Tags could be automatically applied?
    a) True
    b) False

3. Row access policies are unique per table
    a) True
    b) False

4. What are typical use cases for using ACCESS_HISTORY view. Select all correct answers
    a) Satisfy regulatory requirements
    b) Limit the access to data
    c) Performance optimization
    d) Auditing of external access (BI ELT tools, data sharing)

Dynamic Data Masking



Column level security

Dynamically mask data at query run time

Centralized management

Apply one policy to many columns

Dynamic Data Masking Details

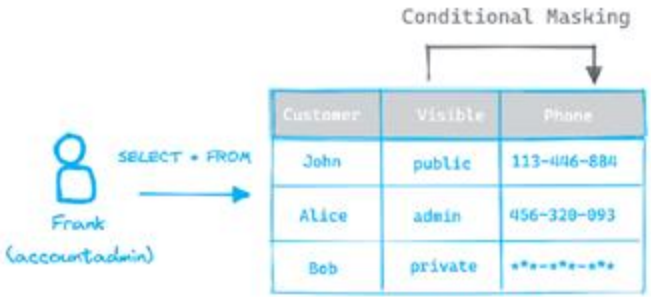
- Can be assigned at creation of table/view or later
- Policy management should be done by security/privacy officers - create a special role for policy management
- Input column data type and masked data type must matched
- Benefits
  - Ease of use
  - Change management
  - Data administration and segregation of duties

```
create or replace masking policy email_mask as (val string) returns string ->
  case
    when current_role() in ('ANALYST') then val
    else '*********'
  end;
```

```
                    case
                      when current_role() in ('ANALYST') then val
                      when current_role() in ('SUPPORT') then regexp_replace(val,'.+\@','*****@')
                      else '********'
                    end;
```

Conditional masking

- Mask data based on value from different column
- No change to stored data
- Unmask them only for authorized users
- Current masking policies could be extended

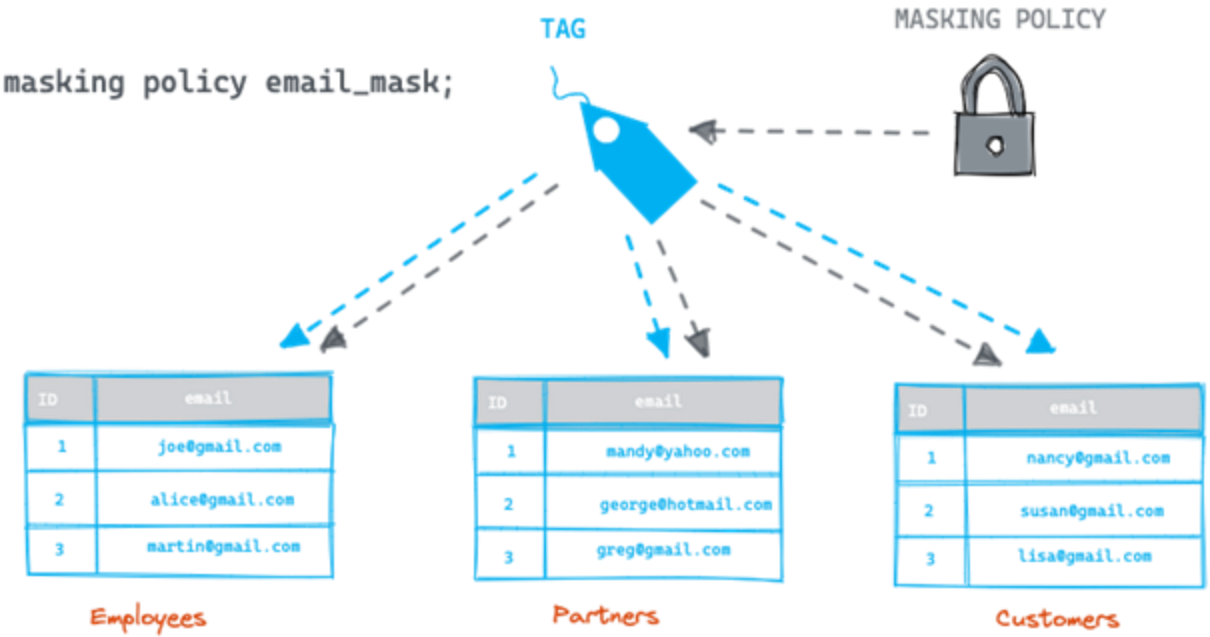



```
CREATE MASKING POLICY conditional_mask_phone
AS
(val STRING, visible STRING) RETURNS STRING ->
CASE
   WHEN current_role() = 'ACCOUNTADMIN' and visible='admin' THEN val
   WHEN visible='public' THEN val
   ELSE '******'
END;
```

Tag based masking

Mask columns automatically based on tags associated with them

```
alter tag personal_email set masking policy email_mask;
```



Limitations:

- One masking policy per data type per tag
- Neither tag nor masking policy can be dropped when masking policy is assigned

Exercise

Let's create a dynamic data masking policy to mask Personal identifiable information (PII) in `TRIPS` table.

Values in `BITH_YEAR` and `GENDER` columns should be visible only to admin roles (`SYSADMIN`, `SECURITYADMIN`, `ACOUNTADMIN`) and `ANALYST` role. All others should see masked value. Let's use 0 as masked value.

1. Create a masking policy called `MASK_PII`
2. Mask data in `BIRTH_YEAR` and `GENDER` columns
3. Test it out - try to query the table as `ANALYST` or `SYSADMIN`. You should see unmasked data. Then change the role to `DEVELOPER` and those columns should be masked

Exercise #2
If you managed previous exercise quickly or you want more practice you can try similar thing but this time with usage of tags. Masking policy should be automatically applied to columns which have assigned given tag.

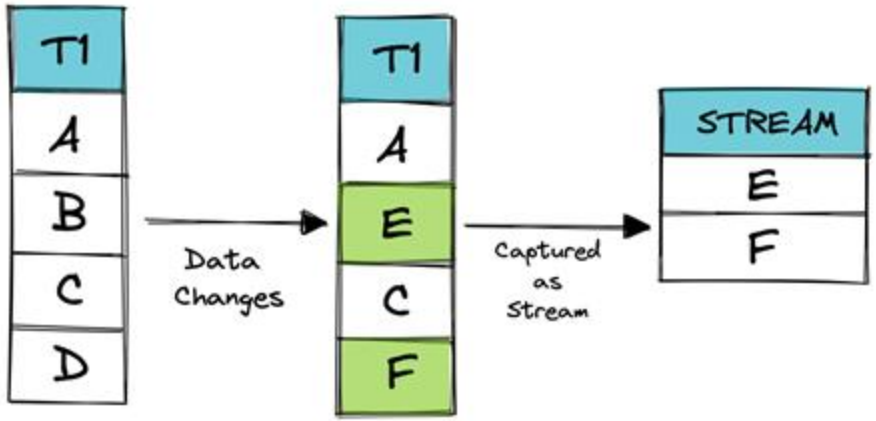
1. Unset the masking policy from `BIRTH_YEAR` and `GENDER` columns
2. Switch to `ACCOUNTADMIN` role
3. Create a tag called `SECURITY_LEVEL`
4. Assign the tag to `BIRTH_YEAR` and `GENDER` columns with tag value = 'PII'
5. Add the `MASK_PII` masking policy to the `SECURITY_LEVEL` tag
6. Test it out - Query the table and used different roles. With `DEVELOPER` role the data should be masked.

O'REILLY®

Streams and tasks

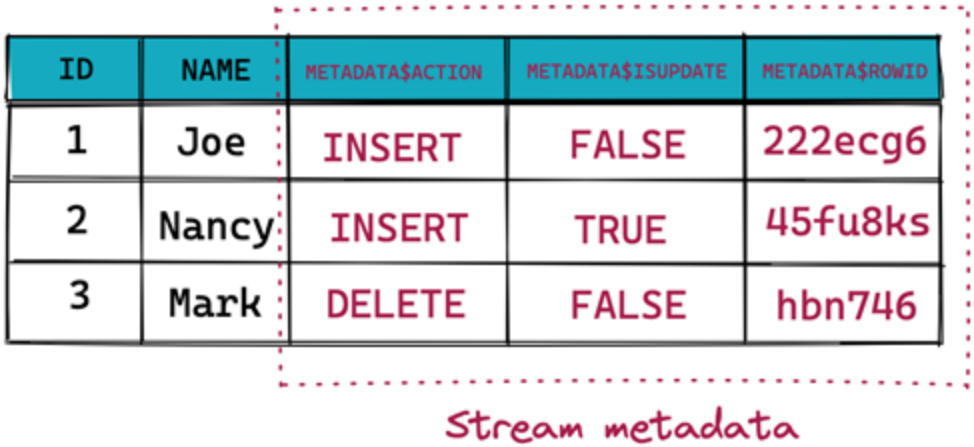
Tomas Sobotik

Streams



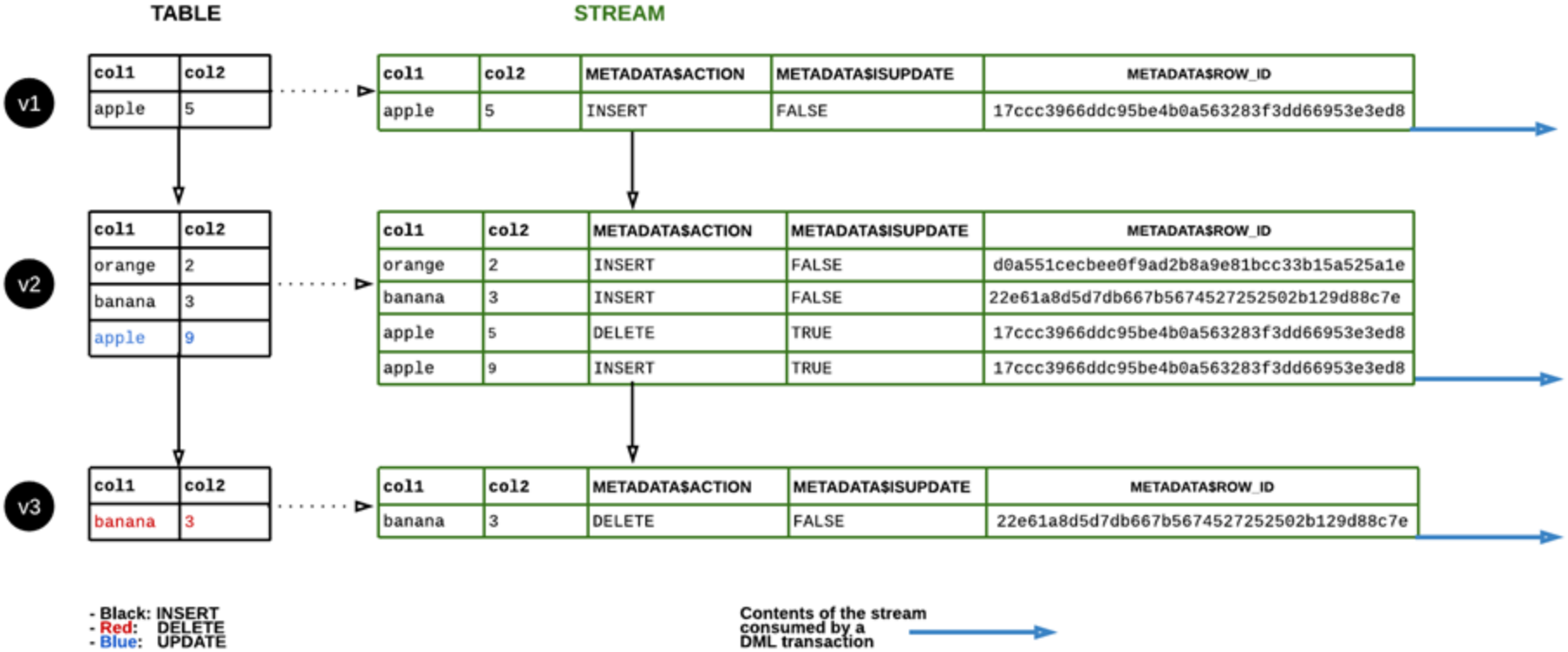
- Changed data capture (CDC)
- Identify and act on changed records
- Stream append 3 metadata columns to the table
  - Tracks the changed data
  - Little storage required
- Can be queried (consumed) as standard table/view
- When consumed as part of DML operation, the stream is cleared
- Stream itself does not contain any data
- Offset = point in time

Streams



- Stream metadata columns
  - METADATA$ACTION
  - METADATA$ISUPDATE
  - METADATA$ROWID
- Type of streams
  - Standard (delta)
  - Append-only
  - Insert-only (external tables)
- Supported objects to track changes
  - tables
  - directory tables
  - external tables
  - views (in preview)

Stream data flow



Data retention and staleness

- If stream is not consumed within defined period of time it becomes stale and data inaccessible
- Default value: 14 days
- Controlled by two parameters
  - DATA_RETENTION_TIME_IN_DAYS
  - MAX_DATA_EXTENSION_TIME_IN_DAYS

- How to check?
  - DESCRIBE STREAM or SHOW STREAMS
    - STALE and STALE_AFTER columns

- When stale => recreate the stream

Streams wrap up

- Multiple consumers needed?
  - Create individual streams per consumer
- Main costs associated with processing time used by virtual warehouse to query the stream
- Think about streams as a “bookmarks”

```
create stream mystream on table mytable;
```

Exercise 1 – Practise the streams

We are going to practice working with streams and tasks. For that we need to prepare some data. Let's simulate we are receiving the data with trips details on monthly basis. We will create a new table called `TRIPS_MONTHLY`, load there some data and build some aggregation logic on top of the table. It will be using streams to track the changes. Later we will combine it with task to automatically process the records when new data arrive into the table.

1. Create a `TRIPS_MONTHLY` table as a copy of `TRIPS` but it will be empty:

```
create table trips_monthly like trips;
```

2. Create a stream on top of `TRIPS_MONTHLY` table
3. Check the stream details (`SHOW STREAMS` command)
4. Check if stream already has some data (system function called `SYSTEM$STREAM_HAS_DATA()`)
5. Let's insert data into our new `TRIPS_MONTHLY` table. We will use data from May 2018. Run following query to load the data from `TRIPS` table. You should load 1 824 658 rows.

```
insert into trips_monthly
        select * from trips
                where date_trunc('month',
starttime) = '2018-05-01T00:00:00Z';
```

6. Now the stream should have data. Check it again (`SYSTEM$STREAM_HAS_DATA()`)

Exercise 1 – Practise the streams

7. Try to query the stream like a normal table. You can find the stream metadata columns at the end of the table (last columns).
8. You can check what kind of unique stream action you have in the table.
9. We are going to consume the records from stream. We will be aggregating the records and storing the results in new fact tables for rides. Let's create such table with following script.

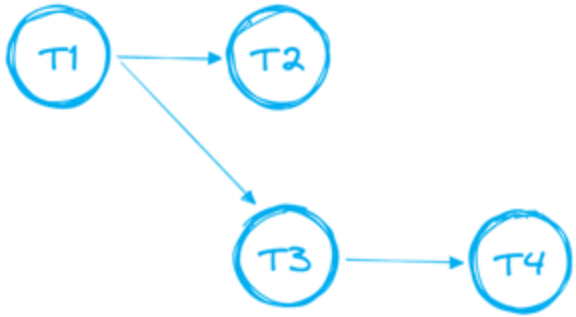
```
create table fact_rides (
                            month timestamp_ntz,
                            number_of_rides number,
                            total_duration number,
                            avg_duration number
);
```

10. Write the aggregation query which would calculate the `NUMBER_OF_RIDES`, `TOTAL_DURATION`, `AVG_DURATION`. As a source you should use the stream `STR_TRIPS_MONTHLY`. Store the result in the new `FACT_RIDES` table.
11. Now the stream should be empty again because we consumed the records in previous step. Let's check it again:

    `SYSTEM$STREAM_HAS_DATA() function should return FALSE`
12. Send into the chat what is `TOTAL_DURATION` and `AVG_DURATION` for all the trips in May 2018

Tasks

- Scheduled execution of SQL operation
  - Single SQL statement
  - Call to a stored procedure
- Often combine with streams for ELT pipelines to process changed table rows
- Trigger
  - Schedule (CRON)
  - Interval (minutes, seconds ..)
  - Predecessor - child task starts when parent task completes
  - Condition = stream contains a new data
  - Manually
- Tasks can be chained together to create complex data pipelines (DAGs) - tree of tasks



Tasks - use cases

Copy data in/out of Snowflake

Complex ELT pipelines

Generate periodic reports

internal cleaning

Keeping aggregations up to date

???

Task creation

- Key parameters
  - WAREHOUSE
  - SCHEDULE
  - WHEN
  - AFTER
  - SUSPEND_TASK_AFTER_NUM_FAILURES

```
create or replace task my_task
   warehouse = xsmall_vwh
   schedule = '1 minute'
   as
    insert into dim_customers (
        select customer_id, customer_name
        from s_src_customer_stream
    where metadata$action = 'INSERT');
```

- Newly created task is suspended
  - Nneeds to be resumed by ALTER command

Task triggered by new data in stream

```
create or replace task my_task
   warehouse = xsmall_vwh
   schedule = '5 minute'
   when SYSTEM$STREAM_HAS_DATA('s_src_customer_stream')
   as
    insert into dim_customers (
        select customer_id, customer_name
        from s_src_customer_stream
    where metadata$action = 'INSERT');
```

Task history retrieval

- TASK_HISTORY view or table function
- Table function = last 7 days history
- View = last year history

- Key columns
  - NAME
  - STATE
  - QUERY_TEXT
  - ERROR_MESSAGE
  - ROOT_TASK_ID

```
select *
  from table(information_schema.task_history(
    scheduled_time_range_start=>dateadd('hour',-1,current_timestamp()),
    result_limit => 10,
    task_name=>'MYTASK'));

 select *
 from snowflake.account_usage.task_history
 limit 10;
```

Exercise 2 - task creation

Now we are going to turn the logic into the task. We are going the create a task which will take records from the `STR_TRIPS_MONTHLY`, do the aggregation and store the results into `FACT_RIDES` table. Task should be scheduled to run every minute but the code will be triggered only when new data will be placed into the stream.

1. Write a task `T_RIDES_AGG` with logic and trigger described above.
2. Check the task definition (`SHOW TASKS`)
3. Resume the task. Newly created tasks are suspended and they have to be resumed in order to start operate.
4. Load May 2018 trips data into `TRIPS_MONTHLY` table.
5. Check the `FACT_RIDES`. New row should be inserted there by our task/

Error notifications for tasks

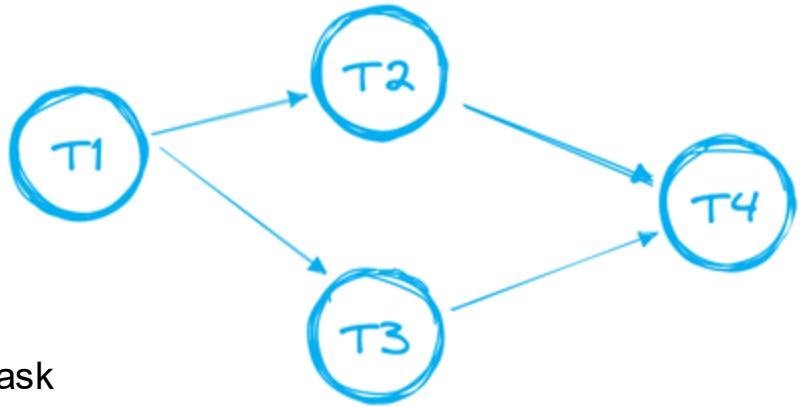
- Receive a notification when task fails
- Snowflake and Cloud Provider integration (AWS, Azure, GCP)
- new Snowflake object **NOTIFICATION INTEGRATION**
  - in preview
  - currently supports only AWS SNS

```
CREATE TASK <name>
  [ ... ]
  ERROR_INTEGRATION = <integration_name>
  AS <sql>
```

Complete step by step guide & in detail overview:
https://medium.com/snowflake/error-notifications-for-snowflake-tasks-ca5798884e67

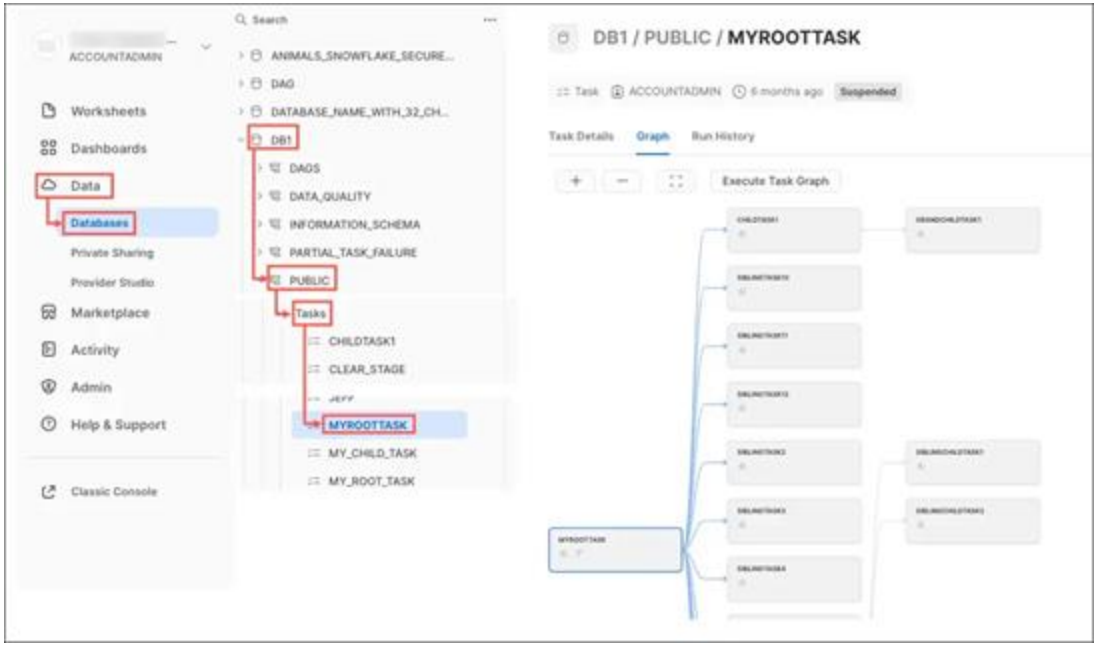
Tree of tasks details

- One root task
- Only single direction - no loop support
- Total numbers
  - 1000 tasks in total for single DAG
  - 100 predecessors tasks
  - 100 child tasks
- If any task in the tree needs to be updated, the root task needs to be always suspended



- How to check the dependencies between tasks
  - TASK_DEPENDENTS table function
  - New UI

New UI for viewing DAG and task history



Streams and tasks best practises

- More stream consumers = multiple streams
- Create a separate custom role for managing the tasks (TASK_ADMIN) with EXECUTE_TASK privilege
- Have a robust operation system around the pipelines
  - Stream stale prevention
  - Task failure notifications
  - Task suspension notifications
  - Pipeline overview
- Version control (GIT integration)
- Do not forget to resume the task

Exercise 3.

Let's create another task which will be running after T_RIDES_AGG. We would have a data pipeline consisting of two steps. Suppose we need some custom logging of the latest loaded month into fact table. For the sake of simplicity we will be just taking the latest loaded month from FACT_RIDES and load it into our custom logging table called LOG_FACT_RIDES. New task should be linked to the T_RIDES_AGG and run after it. T_RIDES_AGG will become a root task of our pipeline.

1. Create a log table:

```
create or replace table log_fact_rides
(
max_loaded_month timestamp_ntz,
inserted_date timestamp_ntz
);
```

2. Before we can create a new task which will be linked to the `T_RIDES_AGG` we need to suspend it first: suspend the `T_RIDES_AGG` task

3. Create a `T_RIDES_LOG` task:
   - takes max loaded month from `FACT_RIDES` table and store it into `LOG_FACT_RIDES` together with current timestamp
   - run after `T_RIDES_AGG`

4. Resume both tasks

5. Load the June 2018 data into the `TRIPS_MONTHLY` table

6. Check the `FACT_RIDES` and `LOG_FACT_RIDES` tables. Both should be populated with data from the latest month.

7. Familiarize yourself with new UI related to tasks. You can open the task details page to see the definition, last runs and much more including the DAG visualisation - how the pipeline looks graphically. Go through those pages to see what everything is available here.

O'REILLY®

Data Sharing

Tomas Sobotik

Snowgrid

- Unique, Snowflake technology
- Connecting regions and clouds together
- It allows:
  - Maintain global business continuity
  - Share data with no ETL, silos
  - Share logic and services
  - Cross-cloud governance controls

AWS Azure GCP

Secure Data Sharing

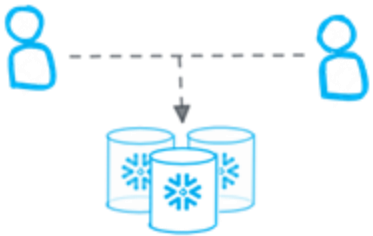
Traditional methods

APIs | FTP | email | ETL |
Cloud storage

- Copy and move data
- Data is delayed
- Costly to manage and maintain
- Unsecure, once data is moved
- Error prone, pipelines break

Snowflake

Secure Data Sharing

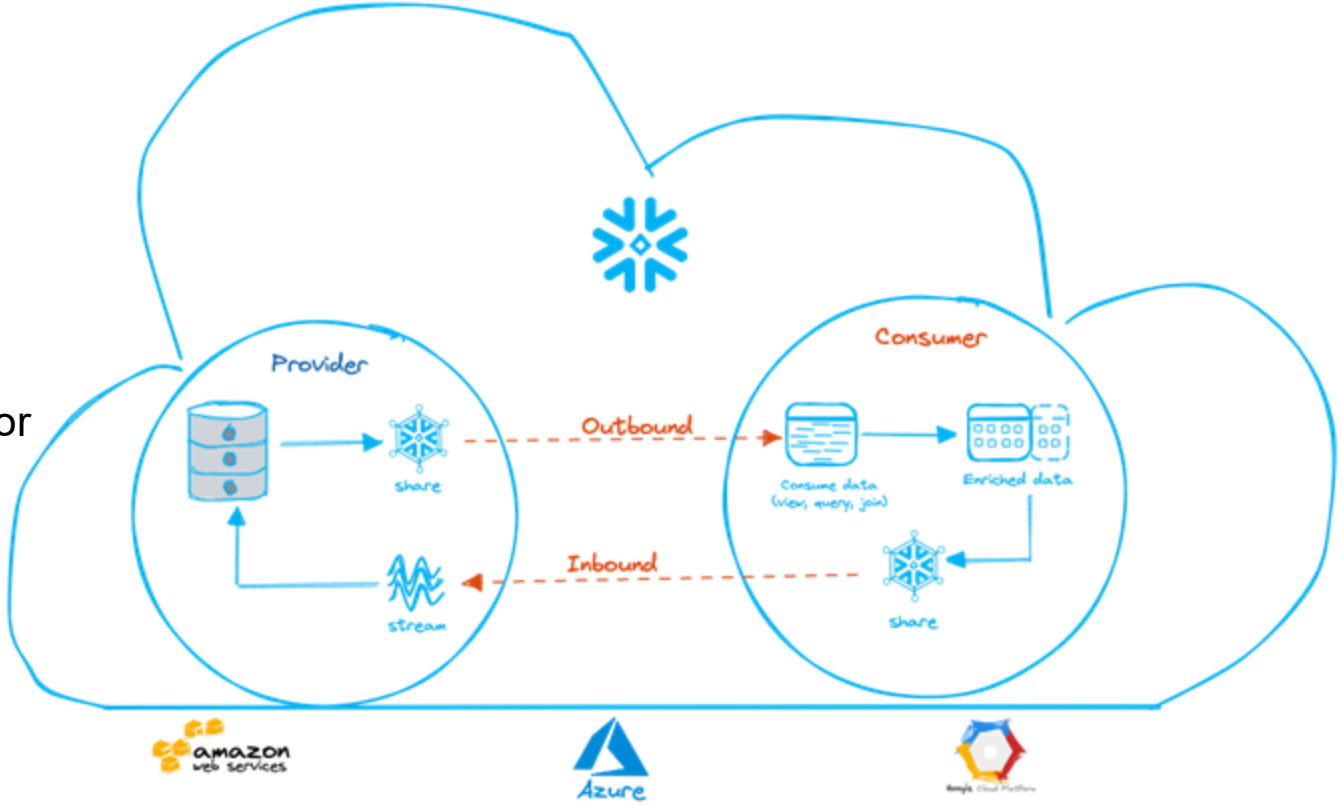


- Single copy of live data, no delays
- No costs of moving, copying, ingestion
- No more data lake silos
- Privacy compliant
- Governed, revocable access

Secure Data Sharing

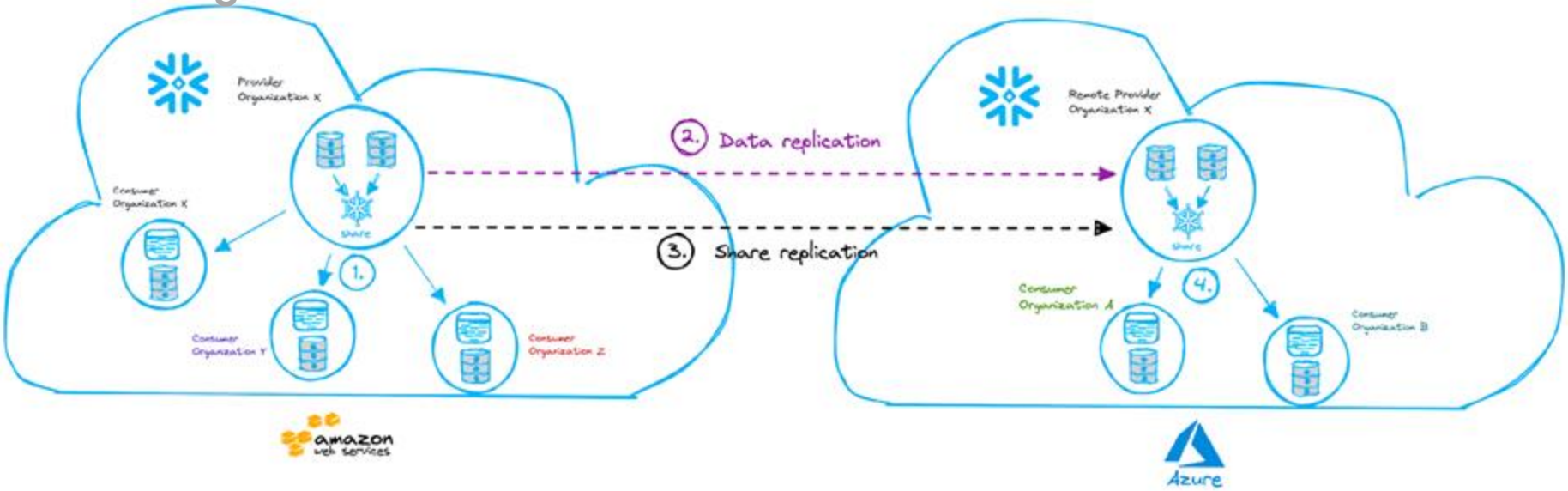
Within a Snowflake region on any cloud

- No data movements
- No data latency within same cloud region
- Provider can instantly revoke object access or drop share
- Shared table/view stream
- For CDC, snapshots



Secure data sharing

Cross region



1. Provider Organization X shares with consumers X, Y, Z in eu-west-1
2. Provider replicates sharted DB to remote provider (must be same org) in Azure West US
3. Provider replicates share definition to Remote Provider (*preview feature)
4. Remote provider shares with Consumers A, B. Task refresh data and share at desired interval

Data sharing types

Data Marketplace

Publish-subscribe model

dataset sharing

free or paid access

utilize Snowsight

Direct share

DB objects sharing

direct connection between provider and consumer

reader accounts

Data exchange

private marketplace

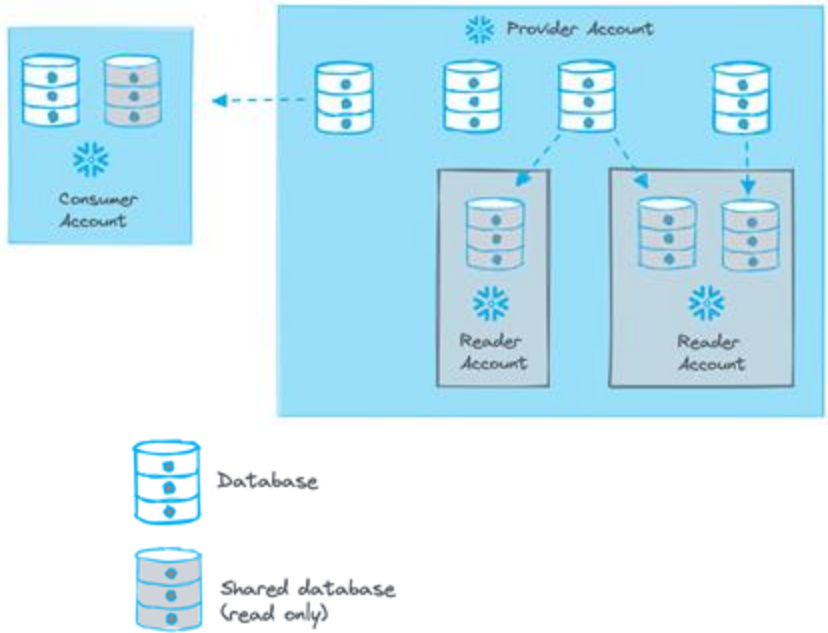
invitation based

provider is in control who can access data

Sharing data with non Snowflake customer

Reader Accounts

- Data provider can create special account
- No requirement for consumer to become Snowflake customer
- Account is owned by data provider
- Credit charges goes to provider account
- Provider share data with reader account
- Reader account can only consume data from provider account - adding data to reader account is not supported



Share components

Tabl
e

External
table

Secure
View

Secure
MV

Secure
UDF

Tas
k

Snowpipe

Stream

Stage

Data share creation - provider side

- To share data it is needed to create a SHARE
  - Snowflake named object
  - Encapsulate all needed information required to share a data
    - Privileges to underlying data (db, schemas, tables, etc.)
    - Consumer accounts

1. Create a SHARE …just a container
2. Grant needed privileges to SHARE
   - USAGE on DB and SCHEMA which should be share
   - SELECT on specific objects which should be shared
     - Tables, secure views, external tables ….
3. Add one or more account into the share
   - ALTER SHARE

Data share creation – consumer side

- Shared data (DB) is read only
- It is not supported:
  - Create a clone of shared DB or any part of it
  - Time travel for a shared DB
  - Editing the Comments for shared DB

1. Viewing available shares - SHOW SHARES or UI
2. Create a database from a Share
   - UI -> Shares -> Inbound
   - SQL -> CREATE DATABASE <name> FROM SHARE <provider_account>.<share_name>

3. Grant shared DB to other roles
   - IMPORTED PRIVILEGES ON DATABASE

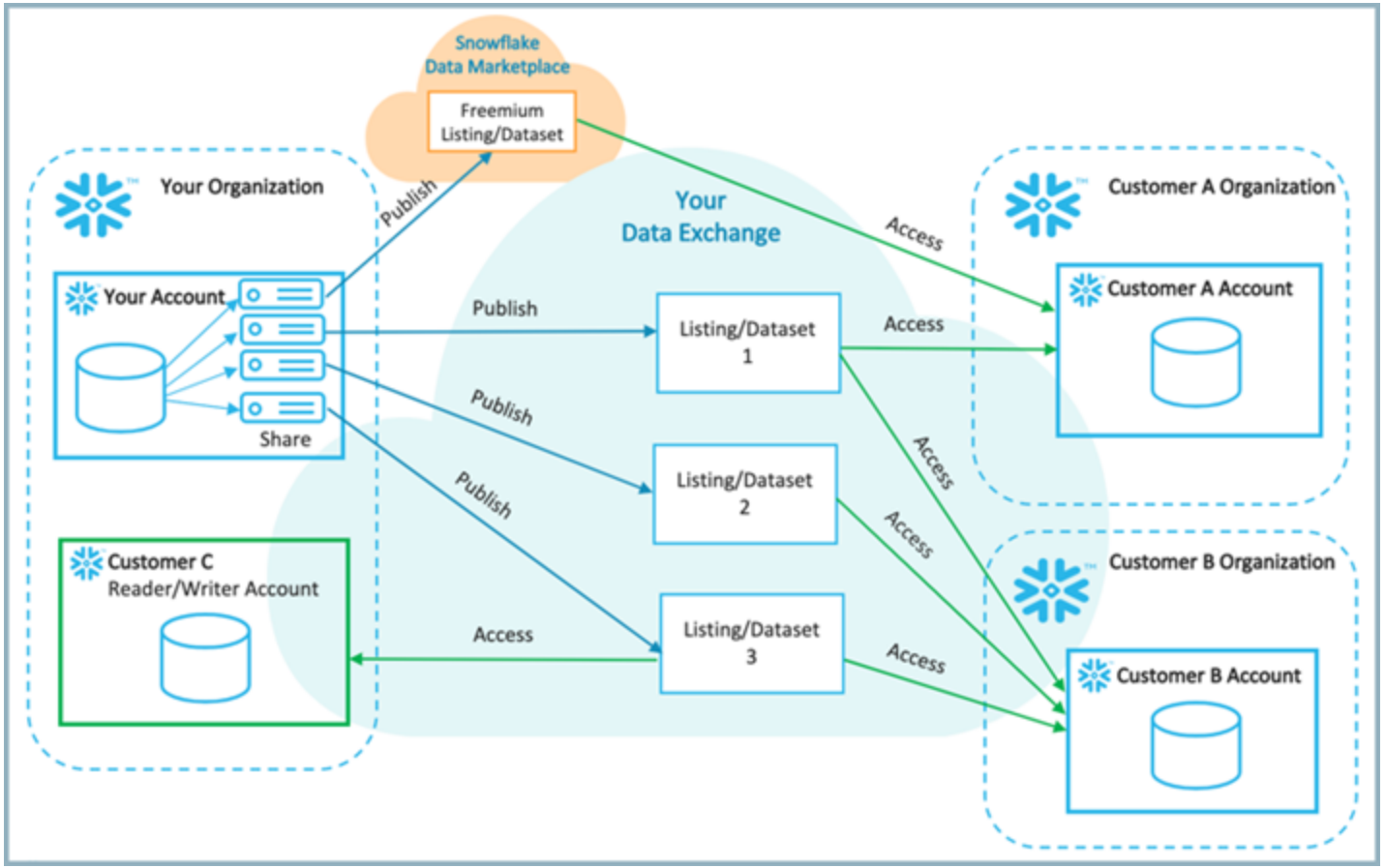
Data share cost

- No additional cost for shares
- Single copy of data
- No data movement
- Only associated cost is related to queries
- Reader accounts = credits are billed to data provider

Data Exchange

- Private data marketplace
- You publish the data and invite consumers
- Use cases
  - Internal data exchange between business units
  - Collaborate with external parties - vendors, partners or customers
- Membership is managed by data provider
- Roles
  - Data Exchange Admin
  - Data Providers
  - Consumers
- Data listings = shared data
- Readers accounts are not supported

Data Exchange



Data Sharing summary

- Modern way how to share data
- Cross cloud, cross region data sharing thanks to Snowgrid
- Breaking down the data silos
- Single copy of the data
- No data movement, copying
- Easy central governance
- Access can be revoked anytime
- Data could be shared with non Snowflake customers
- Different ways how data could be share to support various needs

Exercise

Let's practice creation of the direct data share and all the admin work around (granting needed privileges). There are two options how it can be done. Either there is created a DATABASE ROLE and all the objects which should be part of the share are granted to that DATABASE role, or DB objects are directly granted to the share. Each option has its own use cases where it should be used. You can find more details in documentation.

We are going to try both methods.

Exercise #1 - Data sharing through DATABASE ROLE

1. Create a new database role `SHARE_PROVIDER1` inside `CITIBIKE` DB
2. Grant needed privileges to the new DB role (usage on DB & SCHEMA),
3. We want to share the `TRIPS_MONTHLY` table - grant select to our new DB role
4. Create an empty share `S_MONTHLY_TRIPS`. Do not forget that shares could be created only by ACCOUNTADMIN
5. Grant usage on `CITIBIKE` DB to our newly created share - that's needed prerequisite in order to have access to objects inside
6. Grant the DB role to the share

Now everything what is granted to DB role `SHARE_PROVIDER1` is part of the `S_MONTHLY_TRIPS` data share.

Last step would be adding the consumer's account to actually share the data with someone. That would be done by `ALTER SHARE` command.

Exercise #2 - Data sharing - granting privileges directly to a share

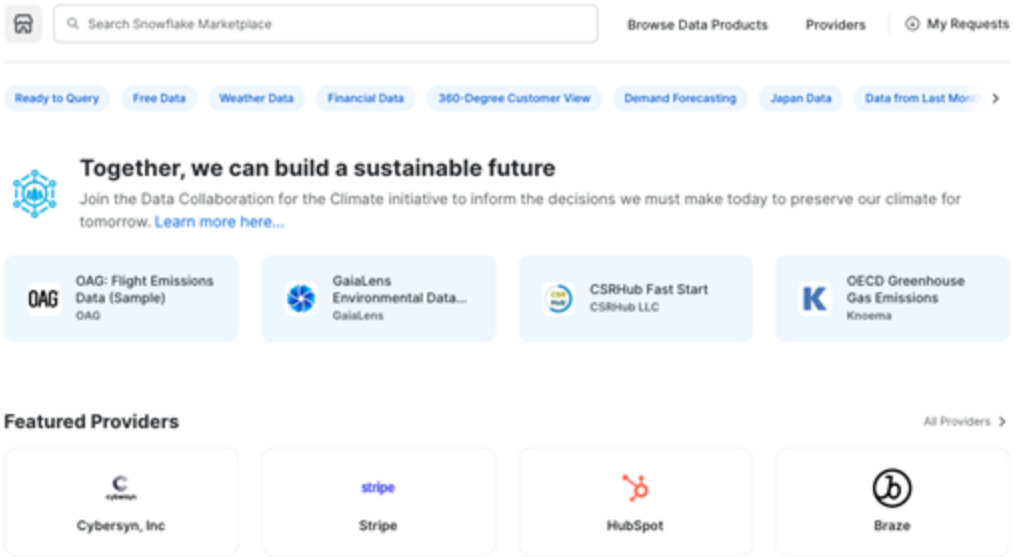
1. Create empty share S_JSON_DATA
2. Grant needed privileges on the share (usage on DB and schema)
3. This time we want to share our JSON data which means JSON_SAMPLE and JSON_TRIPS_PER_STATION tables. Grant SELECT to share

O'REILLY®

Data Marketplace

Tomas Sobotik

Monetize your data



Discover data, services and apps from 260+ providers across 18+ categories

Access the most current data available and receive real time updates automatically

Reduce data integration costs with direct access to live, ready-to-query data; No ETL

Core Concepts

What we already know

- **Provider**
- **Consumer**
- **Share**
- **Region**

Data Product:

- Curated collections of tables, views, UDFs, etc.

Listing:

- Data products enriched by metadata to help make them discoverable
- Type of Listings:
  - Free
  - Monetized - you choose the method of charging the customers. Free sample is provid
  - Personalized - Consumer need to request access before they can use it

Replication:

- Process of duplicating the data from one Snowflake region to another

Pricing models

Per month charge

- Fixed amount per month
- At least one query access the paid data in given month

Per month charge

- Consumer is charged for each query
- Could be combined with per month charge

Maximum total charge per month

- Can be defined how much could be charged per month
- All other queries are free till end of month

Number of free queries

- Free queries per month before per-query charge is applied

Data Providers

- You need to be approved by Snowflake
- You need to provide
  - Fresh data - near real time or updated regularly
  - Real data - not samples
  - Legally shareable - provider must own the data and have right to share it
- Create a provider profile with company details
- Trial accounts can’t be data providers

How to get data from Marketplace

- Everyone can browse listings
- Only ACCOUNTADMIN can get the data
  - IMPORT SHARE privilege can be granted to other roles

Data Marketplace walkthrough

Data Marketplace - summary

- The way how to monetize your data
- Live access to the data
- Cross cloud availability
- Publish them on public data marketplace
- Data Sharing offering
- Providers - publish their datasets
- Consumers - browse marketplace and buy the data
- Datasets could be free or paid

Exercise #1 - Starbucks

We are going to get some free dataset from Data Marketplace. Let's try to add a dataset with Starbucks locations and check if we can combine it with our Trips data.

1. Get the SafeGraph - Free POI Data Sample: US Starbucks locations datasets and store id in `STARBUCKS_LOCATION` DB. Apart from `ACCOUNTADMIN` also `SYSADMIN` and `DEVELOPER` roles should have access to that DB.
2. Go and check the `CORE_POI` table from this marketplace dataset. You can try to find out how many Starbucks from NY are there. Send me the count into the chat!

As we have latitude and longitude and we have the same attributes in our TRIPs data set it would be possible to find out how far are the Starbucks cafés from our Citibike stations. For instance we have a Starbucks branch on following street_address: 1095 Lexington Ave

3. Let's find out how many Citibike stations are located in that street. Send me the number into the chat!

BONUS:
If you would like to play with the data more, and you like the geospatial data. You might try to calculate the distance between citibike stations and starbucks cafés.

Exercise #2 - Weather data

Let's try to add one more data source from Data Marketplace, just to demonstrate how you can easily enrich your data with publicly available datasets. There are plenty of weather related sample datasets. You can try to add some of the Accu Weather datasets to check out how data looks and what attributes it offers.

Which date can you see the forecast for New York? Send me the date to the chat!

O'REILLY®

Snowflake Internal DB

Tomas Sobotik

Snowflake metadata

Usage metrics

ACCOUNT_USAGE

Part of Snowflake database

Views

Up to year of data retention

Include drop objects

Latency up to 3 hours

Available only for ACCOUNTADMIN

INFORMATION_SCHEMA

Part of each database

Table functions

Data retention between 7 days to 6 months

No dropped objects

No latency

Anyone with usage privileges for given DB

ACCESS_HISTORY

Key views from ACCOUNT_USAGE schema

- Get an overview about reads and writes in your Snowflake account
- Satisfy regulatory compliance
- Understand your usage patterns and optimise storage cost and performance
  - Discover unused tables/columns
  - Discover the most frequently used tables for performance optimisations
  - Check out the speed of adoption for new objects
- Discover the data lineage
  - DIRECT_OBJECTS_ACCESSED
  - BASE_OBJECTS_ACCESSED
  - OBJECTS_MODIFIED
- QUERY_ID
- QUERY_START_TIME
- USER_NAME

Columns in JSON array

DIRECT_OBJECTS_ACCESSED, BASE_OBJECTS_ACCESSED, OBJECTS_MODIFIED

Field	
columnId	A column ID that is unique within the account
columnName	The name of the accessed column
objectId	An identifier for the object (TABLE_ID, STAGE_ID, etc.)
objectName	The fully qualified name of the object that was accessed.
objectDomain	Table, view, Materialized view, External table, Stream, Stage
location	The URL of the external location when the data access is an external location
stageKind	When writing to stage - Table, User, Internal Named, External Named
baseSources	The columns that serve as the source columns for the columns for the columns specified by directSources.
directSources	The columns specifically mentioned in the data write portion of the SQL statement.

ACCESS_HISTORY read example

Which queries accessed sensitive table in the last 30 days

```
select query_id
       , query_start_time
from access_history
     , lateral flatten(base_objects_accessed) f1
where f1.value:"objectId"::int=329984114
and f1.value:"objectDomain"::string='Table'
and query_start_time >= dateadd('day', -30,
current_timestamp())
```

Which columns were accessed in the last 30 days

```
select distinct f4.value as column_name
from access_history
     , lateral flatten(base_objects_accessed) f1
     , lateral flatten(f1.value) f2
     , lateral flatten(f2.value) f3
     , lateral flatten(f3.value) f4
where f1.value:"objectId"::int=329984114
and f1.value:"objectDomain"::string='Table'
and f4.key='columnName';
```

ACCESS_HISTORY write example

Loading data from external stage

```
copy into table1(col1, col2)
from (select t.$1, t.$2 from @mystage1/data1.csv.gz);
```

The DIRECT_OBJECTS_ACCESSED and BASE_OBJECTS_ACCESSED column says that external named stage was accessed

The OBJECTS_MODIFIED column says that data was written to two columns of the table


```
{
  "objectDomain": STAGE
  "objectName": "mystage1",
  "objectId": 1,
  "stageKind": "External Named"
}
```

```
{
  "columns": [
    {
      "columnName": "col1",
      "columnId": 1
    },
    {
      "columnName": "col2",
      "columnId": 2
    }
  ],
  "objectId": 1,
  "objectName": "TEST_DB.TEST_SCHEMA.TABLE1",
  "objectDomain": TABLE
}
```

WAREHOUSE_METERING_HISTORY & MEETERING_HISTORY

- Return the hourly credit usage for a single warehouse or all the warehouses in your account
- Retention: 1 year

```
select
    sum(credits_used)
from
    snowflake.account_usage.metering_history
where
    start_time ≥
to_timestamp(dateadd('days', -7, current_timestamp));
```

```
select
    warehouse_name,
    sum(credits_used) as total_credits_used
from
    snowflake.account_usage.warehouse_metering_history
where
    start_time ≥
    to_timestamp(dateadd('days', -7, current_timestamp))
group by
    1
order by
    2 desc;
```

QUERY_HISTORY

- Query history by various dimensions (user, warehouse, time range, etc.)
- Retention: 1 year
- Examples
  - Number of failed queries
  - Query performance
    - Scanned partitions
    - Execution time

Average execution time by query type

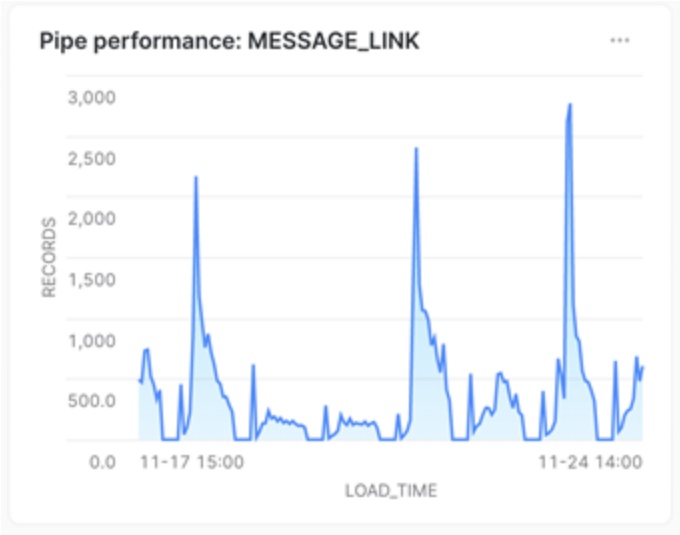
```
select
    query_type,
    warehouse_size,
    avg(execution_time) / 1000 as average_execution_time
from
    snowflake.account_usage.query_history
where
    start_time  ≥
    to_timestamp(dateadd('days', -7, current_timestamp))
group by
    1,
    2
order by
    3 desc;
```

TABLES/VIEWS/COLUMNS/PIPES, FUNCTIONS, …

- List of tables in your account
- Info about dropped objects
- Table size
- Table type
- Time travel settings

Snowsight dashboards

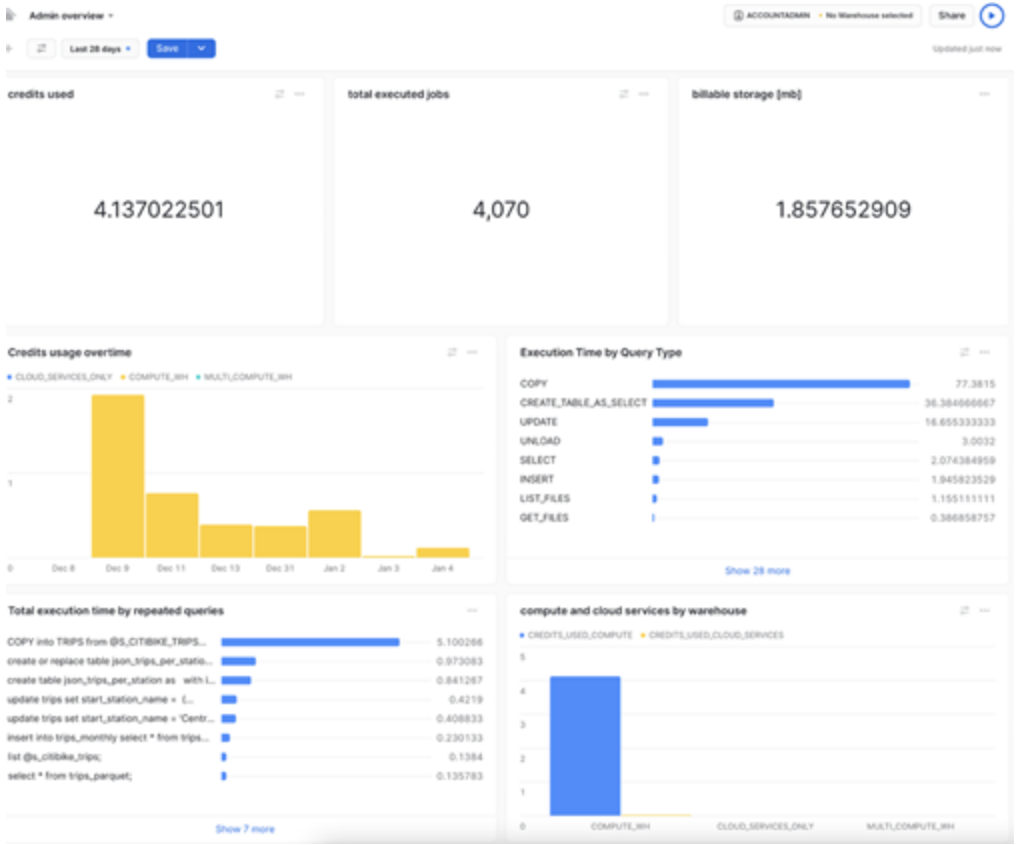
- Query result could be visualized
- Visualizations could be combined into dashboards
- Dashboards could be shared with coworkers
- Drag and drop interface
- Support for filters
- Basic charts available
  - Line
  - Bar
  - Heat grid



Dashboard demo

Exercise

Let’s practise two concepts in this exercise. First will be creation of Snowsight dashboards and second one would be related to using of Snowflake internal database which is full of interesting metadata. We are going to create an Admin dashboard which will be visualising data about our account usage - how many credits we have spent, how many jobs have been performed, what are the longest running queries etc.



Exercise

I will provide some queries for the first objects on the dashboard to help you out with syntax and show case what kind of data you can find in ACCOUNT_USAGE schema.

1. create a new tile from a worksheet. This one will be showing total used credits in given time frame. As a query use following one:

```
select
    sum(credits_used)
from
    account_usage.metering_history
where
    start_time = :daterange;
```

You can notice special syntax in part related to date time filter. With following syntax :daterange you are saying that the real value should be taken from dashboard setting in the runtime.

2. Run the query, toggle in the chart settings and select Scorecard as chart type.
3. Rename the title to **Credits used** and return back to dashboard overview
4. Create a new tile from a worksheet for another dashboard component.
5. This time use following query to get total storage in megabytes

```
select
    avg(storage_bytes + stage_bytes + failsafe_bytes) / power(1024, 3) as billable_mb
from
    account_usage.storage_usage
where
    USAGE_DATE = current_date() -1;
```

Exercise

6. Next steps are same like in the previous example. Toggle in the chart settings, select Scorecard as chart type, rename the worksheet to some more descriptive name and return back to dashboard settings.

7. Last dashboard tile where I will provide a query for you will be for execution time by query type. You can get the data with following query

```
select
    query_type,
    warehouse_size,
    avg(execution_time) / 1000 as average_execution_time
from
    account_usage.query_history
where
    start_time = :daterange
group by
    1,
    2
order by
    3 desc;
```

Exercise

8. Toggle in chart settings, this time select bar chart, select `AVERAGE_EXECUTION_TIME` column as aggregated one with sum operation, then select `QUERY_TYPE` as Y-Axis, switch to horizontal bars and rename the chart to Execution Time by Query Type.

Now it is time to practise working with `ACCOUNT_USAGE` data on your own. Please try to develop the queries for next dashboard charts by yourself. You can try to create some from the following:

9. Create a scorecard number with total number of executed jobs
10. Create a bar chart showing credit usage over time per virtual warehouse
11. Create a bar chart showing used credits for compute and used credits for cloud services per virtual warehouses
12. Create a bar chart with total execution time per query for repeated queries

If you do not know what kind of view you should use, go and check the documentation where you can find out more details about individual views and what kind of data they offer.

O'REILLY®

Programmability in Snowflake

Tomas Sobotik

Running the code in Snowflake

UDF/ UDTF

must return value

scalar & tabular

Stored Procedures

can return value

procedural logic

External Functions

code runs outside of SF

any language support

AWS, Azure, GCP

Snowpark

API for processing data

runs in SF

DataFrame abstraction

User Defined Functions

- Support for SQL, JavaScript, Java and Python
- Develop a reusable logic which is not part of Snowflake
  - Area of circle
  - Profit per department
  - Concatenate names
  - Get bigger number
- Only SQL and JavaScript UDFs can be shared
- DML and DDL is not permitted
- Secure version (data sharing)

```
create function area_of_circle(radius float)
  returns float
  as
  $$
    pi() * radius * radius
  $$
  ;
```

Stored procedures

- SQL, JavaScript, Python,Java
- Bundle multiple SQL commands into single callable script
- Variables, loops, conditions, cursors
- Transaction support
- Error handling
- Dynamic creation of SQL
- Called as independent statements
  - CALL myStoredProcedure(argument);
- One SP can call another one
- Cannot return a set of rows

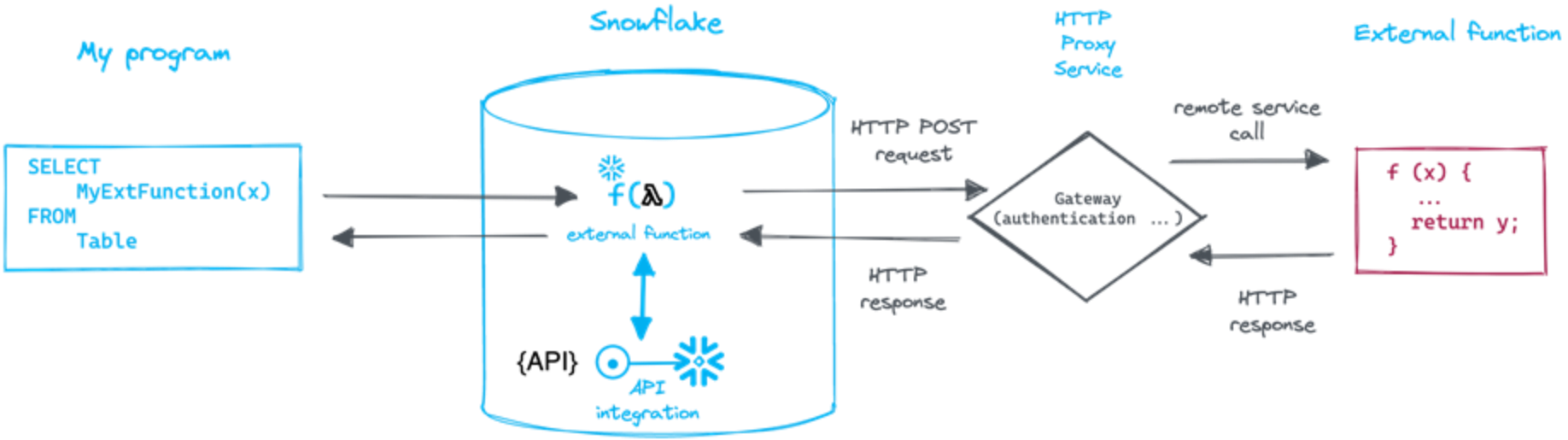
```
create or replace procedure myprocedure()
  returns varchar
  language sql
  as
  $$
    -- Snowflake Scripting code
    declare
      r float;
      area_of_circle float;
    begin
      radius_of_circle := 3;
      area_of_circle := pi() * r * r;
      return area_of_circle;
    end;
  $$
  ;
```

External Functions

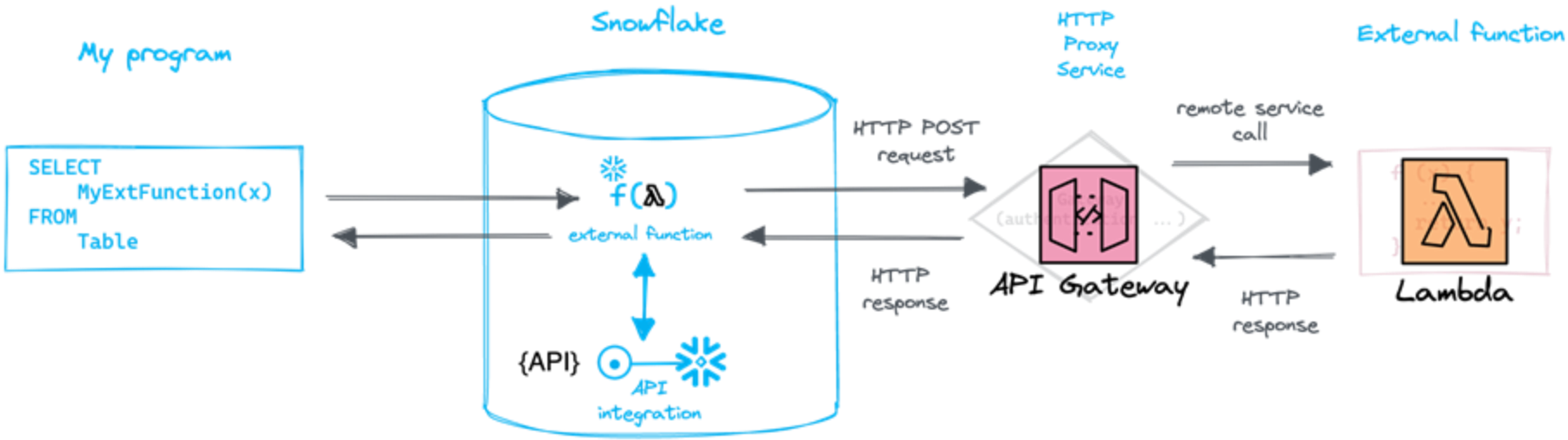
- Call executable code which has been developed and runs outside of Snowflake
- Call external APIs, process data in Snowflake
- No need to export and reimport data
- Simplify your data pipelines
- Communication with external service is done by API integration
- Requires configuration on external cloud platform
- Use cases
  - ML models training
  - Translate service
  - Integration with other tools (Jira, etc.)

```
create or replace api integration demonstration_external_api_integration_01
    api_provider=aws_api_gateway
    api_aws_role_arn='arn:aws:iam::123456789012:role/my_cloud_account_role'
    api_allowed_prefixes=('https://xyz.execute-api.us-west-2.amazonaws.com/production')
    enabled=true;
```

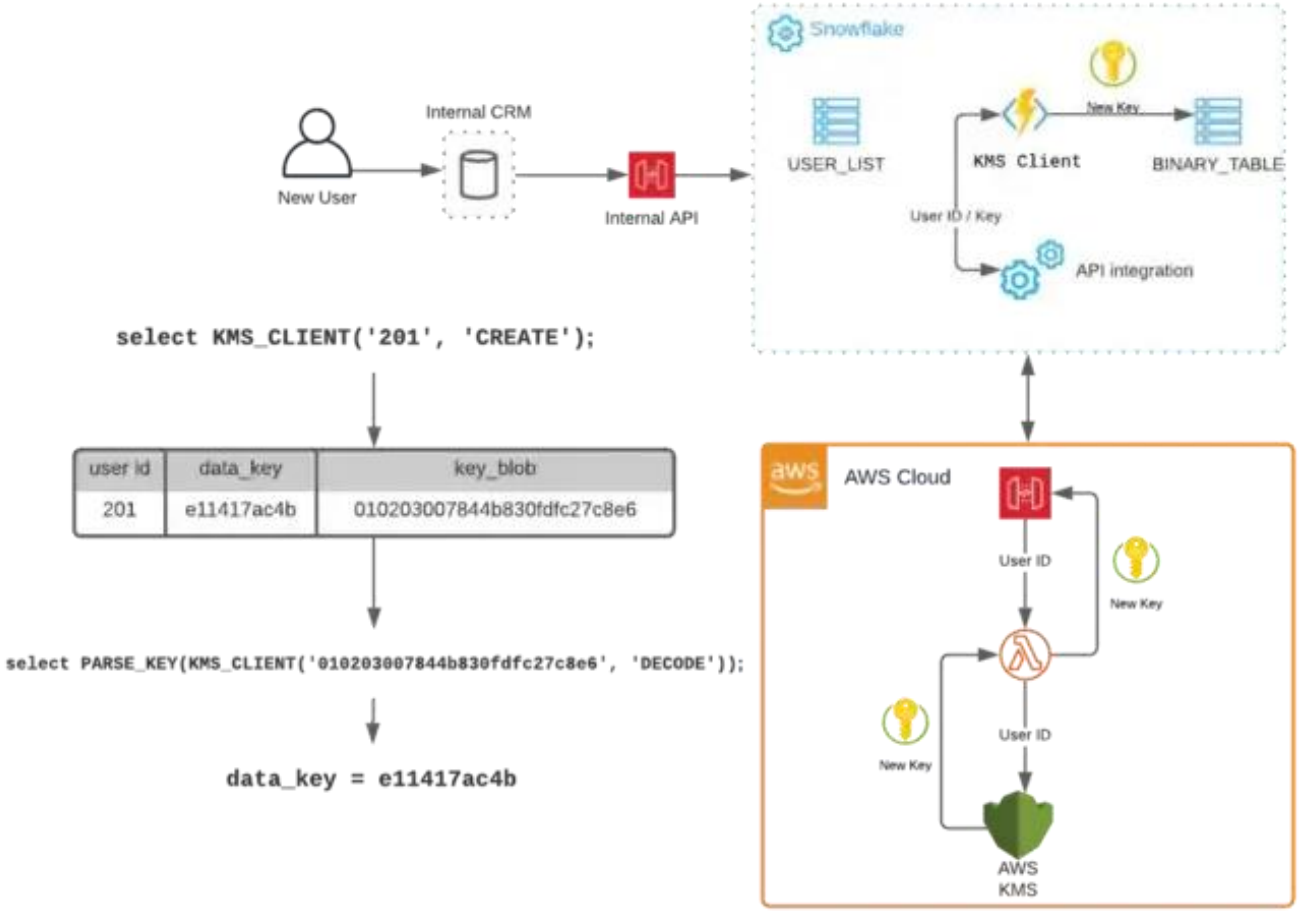
External Function – high level overview



External Function – high level overview



External Function - custom data encryption



Exercise

Let’s practise creation of user defined function (UDF) which will be using SQL as a language. Our trips dataset contains column called `BIRTH_YEAR`. Write a function called `GET_AGE` which will calculate the current age of the users based on their birth year.

Test the function in the SQL. Write `SELECT` statement using your newly created UDF.

If you are done. You can try to create one more function. We need to check if the bike ride was done during weekend or not. Write a UDF called `WEEKEND_RIDE_CHECK` which will return True in case the bike ride was done during weekend (Saturday or Sunday) otherwise it returns False.

O'REILLY®

Serverless features

Tomas Sobotik

What is serverless?

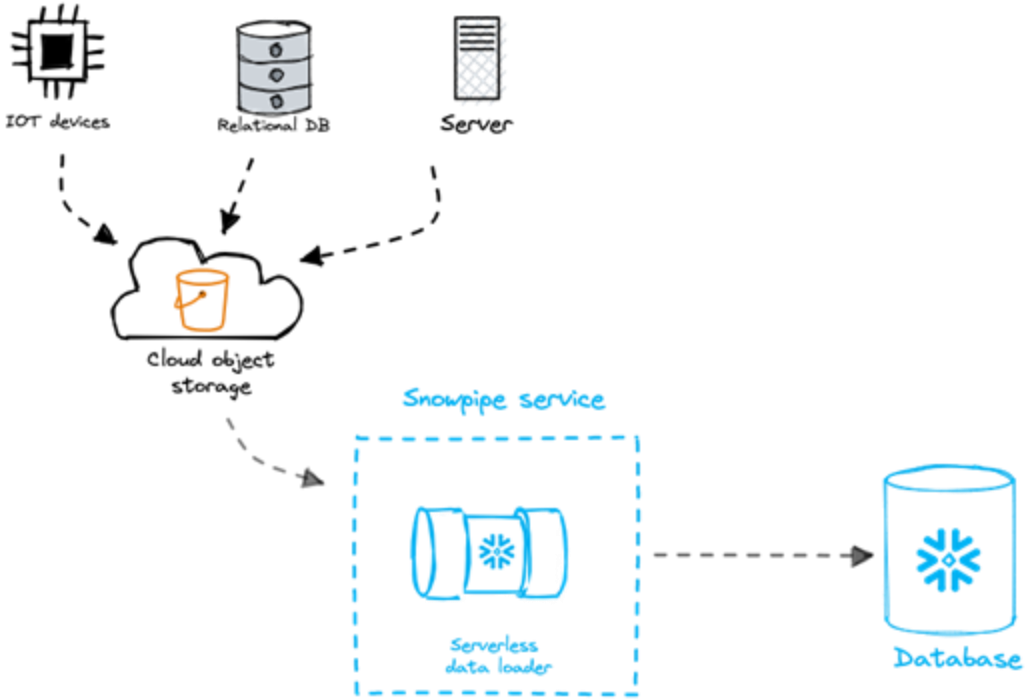
> “Focus on your **application**, not the **infrastructure**”

- Infrastructure is provisioned by cloud provider
- What does it mean in Snowflake world?
  - No need to take care about Virtual Warehouses

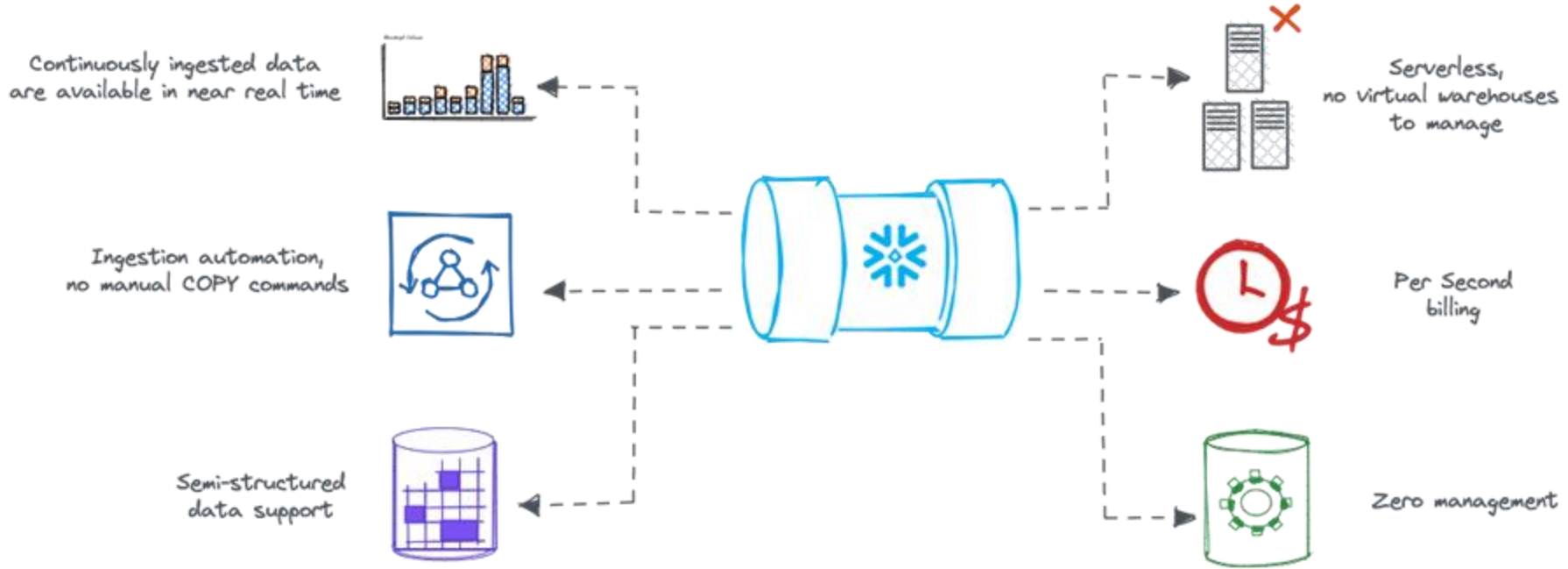
- Serverless features
  - **Snowpipes**
  - Cross cloud replication
  - Materialized Views
  - Search Optimization
  - Auto clustering
  - Query Acceleration
  - **Serverless Tasks**

Snowpipes

- Automated data loading feature
- Near real time data streaming
- No need to dedicate and suspend virtual warehouse
- Serverless with per second billing
- File (batch) oriented
- Credit consumption is visible under SNOWPIPE warehouse
- Use cases
  - Automated data ingestion
  - IOT data stream import
  - Data Lake files import
  - Kafka - streaming the data directly into Snowflake



Faster data ingestion with Snowpipes



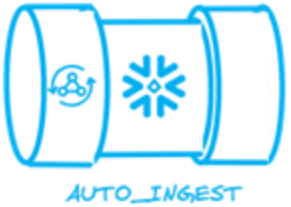
Snowpipes - how it works

- Pre-defined object that specifies the COPY command for data load
- Contains instructions how and where to load and convert data into records in the target table
- Supports all data types (including semistructured)

Manually call Snowpipe REST API endpoint

Pass a list of stage files in the stage location

Works with internal and external stages



Receive notifications from cloud provider when files arrive

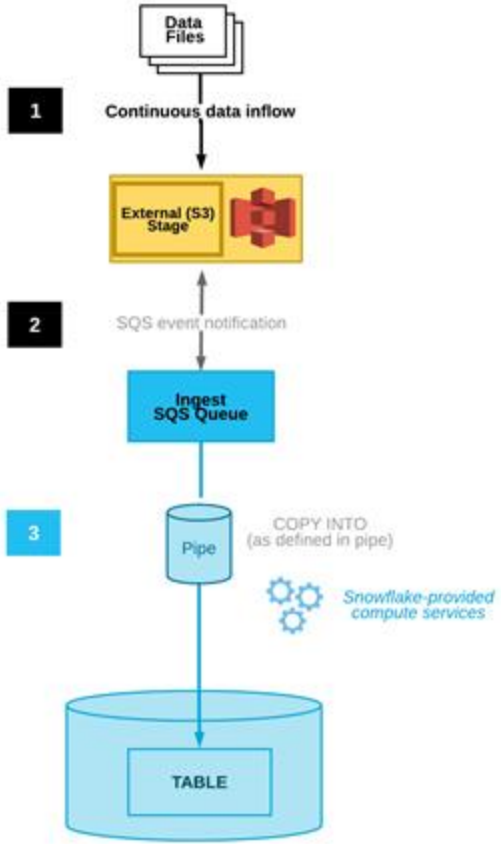
Notification trigger the processing

Only external stages are supported

How to create Snowpipe with Auto Ingest

- Snowflake hosted on AWS support all clouds, rest only their own
- It requires configuration on cloud provider side
  - Bucket access permission
  - IAM role for Snowflake
  - SQS notification for S3 bucket
- Snowflake configuration
  - Storage integration object - encapsulates the access privileges for accessing the S3 bucket

```
create pipe mypipe auto_ingest=true as
  copy into mytable
  from @my_db.public.mystage
  file_format = (type = 'JSON');
```



Snowpipe failures monitoring

- In preview
- New object: NOTIFICATION INTEGRATION
- Snowflake is able to send you a SNS message whenever the snowpipe fails
- Based on received notification you can automate the pipeline monitoring

https://medium.com/snowflake/error-notifications-for-snowflake-tasks-ca5798884e67

Snowpipe demo

Serverless tasks

Task

```
create task t1
  schedule = '1 minute'
  warehouse = 'transform_wh'
  AS …
```

Serverless Task

```
create task t1
  schedule = '1 minute'
  warehouse = 'transform_wh'
  AS …
```

- No need to specify a warehouse - size selection, resume and suspend is done automatically
- Simpler and more cost efficient
- Existing task can be modified to be serverless
- It uses last few executions to decide warehouse size

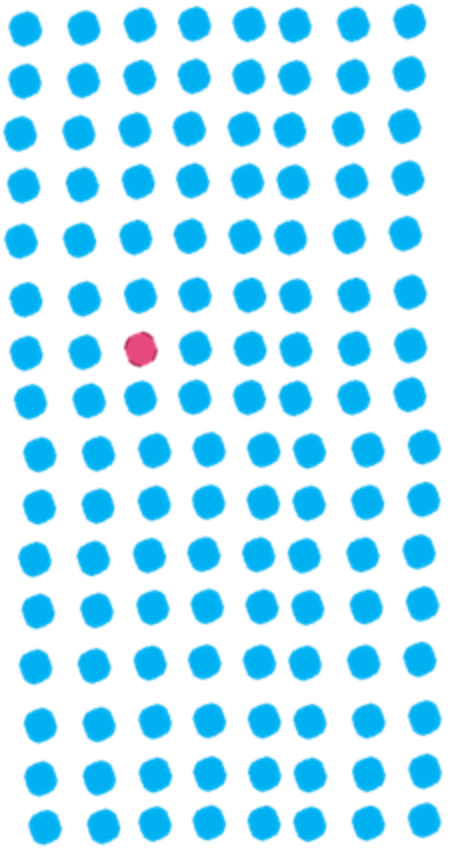
Search optimization service

Find the needle in the haystack

- Lookup queries
- Find a small number of rows in large tables
- Faster query performance on data filtering or data exploration workloads
- Managed service that accelerates point lookup

Use cases

- Highly selective filters in dashboards
- Specific subset of data in large ML datasets



Search optimization service - how it works

- Maintenance service keeps up to date so called search access path
  - In background, not blocking any operation
  - Needs to be updated every time there is new/changed data in the table
- Currently supports - INTEGER, NUMERIC, DATE, TIMESTAMP, VARCHAR, BINARY
  - In preview: VARIANT, OBJECT, ARRAY

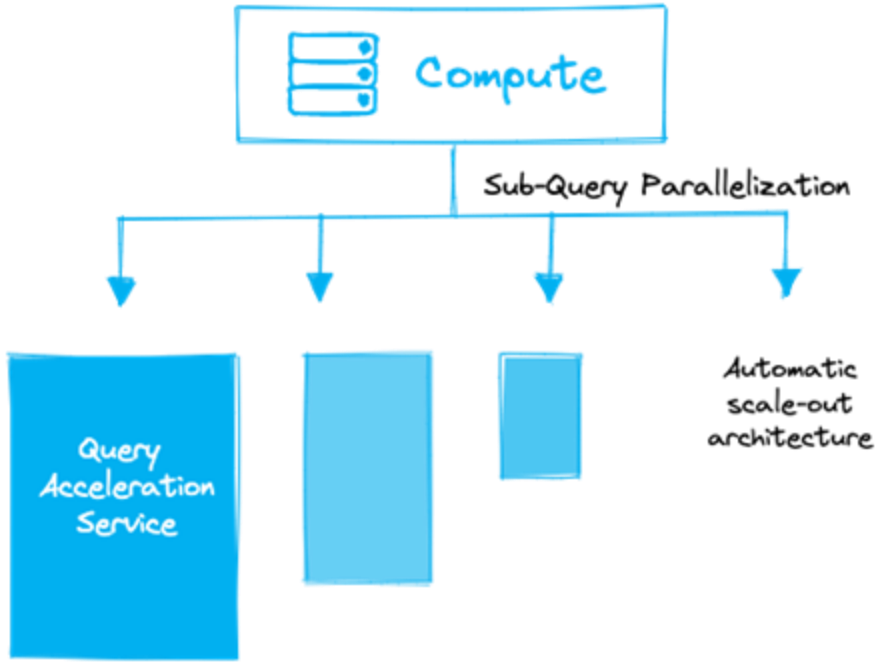
- What type of queries could be improved
  - Equality, OR, IN predicates
  - Substrings and regular expressions (LIKE, CONTAINS, REGEXP, …)
- Before enabling you can get cost estimation **`SYSTEM$ESTIMATE_SEARCH_OPTIMIZATION_COSTS`**
- Could be added on specific columns or entire table

```
Alter table t1 add search optimization;
```

Query Acceleration

Elastic scale out of compute resources without changing size of virtual warehouse

- Performance improvement feature
- Reduce execution times
- Improve performance with existing compute resource for spikes in workload compute needs
- It enables vertical scaling of individual queries by creating sub-query parallelization
- Supports INSERT, SELECT, CTAS
- Use cases
  - Ad-hoc analysis
  - Queries with large scans and selective filters
  - Workloads with unpredictable data volume per query



How to use query acceleration service

1. Identify queries / warehouses which might benefit from using this service
   - `SYSTEM$ESTIMATE_QUERY_ACCELERATION(<query_id>)`
   - `Use QUERY_ACCELERATION_ELIGIBLE view`
     - You can get eligible query acceleration time

```
"estimatedQueryTimes" : {
      "1" : 171,
      "2": 152,
      "4": 133,
      "8": 120
}
```

2. Enable the service on WH level

```
alter warehouse my_wh set
  enable_query_acceleration = true
  query_acceleration_max_scale_factor =
8;
```

2. Monitor the usage
   - Profile Overview
   - `QUERY_HISTORY` view
   - `QUERY_ACCELERATION_HISTORY` view for billing

Exercise - serverless task

We are going to practise the serverless tasks in this exercise. We already have two tasks as part of our project - T_RIDES_AGG, T_RIDES_LOG. Those tasks uses user managed virtual warehouses. Now we turn them into being serverless.

1. At the beginning, let's clean up the tables which tasks use as targets:

```
truncate table fact_rides;
truncate table log_fact_rides;
```

2. Change the `T_RIDES_AGG` task to be a serverless task. Use `ALTER TASK` command.

3. Check the definition of the task. Now the warehouse attribute should be empty. It means that task is serverless

```
show tasks;
```

4. We try to recreate the other task instead of altering it. In order to be able to create serverless task with SYSADMIN role, we need to grant a new privileges - `EXECUTE MANAGED TASK`:

```
use role accountadmin;
grant EXECUTE MANAGED TASK on account to role SYSADMIN;
use role sysadmin;
```

Exercise - serverless task

5. Now we can recreate the task and make it server less by omitting the WAREHOUSE attribute

```
create or replace task t_rides_log
comment = 'Logging the last loaded month'
after T_RIDES_AGG
AS
insert into log_fact_rides select max(month), current_timestamp from
fact_rides;
```

6. Let's test it out now and see what kind of virtual warehouse Snowflake will automatically assign to our task. First we need to resume both tasks

```
alter task t_rides_log resume;
alter task t_rides_agg resume;
```

7. Insert some data into our `TRIPS_MONTHLY` table. Just to repeat. Those data will be part of stream `STR_TRIPS_MONTHLY` and it will automatically trigger our root task and then also the logging task.

```
insert into trips_monthly select * from trips
where date_trunc('month', starttime) = '2018-06-01T00:00:00Z';
```

Exercise - serverless task

8. Let's check the tables to see if the pipeline has finished.

```
select * from fact_rides;
select * from log_fact_rides;
```

9. If both tables contain the data. We can suspend our tasks now.

```
alter task T_RIDES_LOG suspend;
alter task t_rides_agg suspend;
```

10. Now we can check what virtual warehouse was automatically provisioned by Snowflake for our task run. Let's check the task_history table function to find out the task's query_id:

```
select *
  from table(information_schema.task_history(
  TASK_NAME => 'T_RIDES_AGG'
  ))
  order by scheduled_time desc;
```

Exercise - serverless task

11. Find a row with query_id value. It should have also state value = SUCCEEDED. Copy the query_id

12. Let's look into the SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY for query details of given query_id.

```
use role accountadmin;

select * from snowflake.account_usage.query_history where query_id in ('your query id from previous step');
```

You can find in the result set the details about used warehouse. You can see that in my case Snowflake has used the medium size warehouse.

	WAREHOUSE_ID	WAREHOUSE_NAME	WAREHOUSE_SIZE	WAREHOUSE_TYPE
1	54763825	COMPUTE_SERVICE_WH_USER_TASKS_POOL_MEDIUM_0	Medium	STANDARD

Wrap up

- You should know everything what you might need for your next Snowflake project
- Great knowledge foundation for Snowpro Core Certification
- What we have learnt
  - Snowflake architecture principles
  - Virtual Warehouses
  - Billing
  - Storage
  - Data Loading principles
  - Semi structured data
  - Data Sharing
  - Data Governance
  - Serverless features
  - Streams and Tasks
  - etc …

Wrap up

- You should know everything what you might need for your next Snowflake project
- Great knowledge foundation for Snowpro Core Certification
- What we have learnt
  - Snowflake architecture principles
  - Virtual Warehouses
  - Billing
  - Storage
  - Data Loading principles
  - Semi structured data
  - Data Sharing
  - Data Governance
  - Serverless features
  - Streams and Tasks
  - etc …

Good Luck!

Wrap up

- You should know everything what you might need for your next Snowflake project
- Great knowledge foundation for Snowpro Core Certification
- What we have learnt
  - Snowflake architecture principles
  - Virtual Warehouses
  - Billing
  - Storage
  - Data Loading principles
  - Semi-structured data
  - D
  - D
  - S
  - Streams and Tasks
  - etc...

Good Luck!

Please, share your feedback.

Thank you!

https://www.linkedin.com/in/tomas-sobotik/